

Universidad de la Sierra Sur

Integración del evento de Amazon S3 con AWS Lambda, logs, métricas y alarma para la Gaceta Universitaria «El Unsito»

MANUAL DE IMPLEMENTACIÓN

PROYECTO INTEGRADOR · EQUIPO 3
TECNOLOGÍAS DE LA INFORMACIÓN I

Presentan:

Daniel González Ruiz
Mayra Castellanos Pacheco
Gael Eduardo García Pinacho
Elisa Sánchez Martínez

Docente:

M.C.C. Eliezer Alcazar Silva

Miahuatlán de Porfirio Díaz, Oaxaca.
Noviembre de 2026

Contenido

1. Antes de empezar — 2
2. Revisar lo que ya existe — 4
3. Actualizar la función de prefirmadas — 6
4. Crear el rol de la procesadora — 9
5. Crear la función procesadora — 12
6. Conectar S3 con la función — 18
7. Agregar la consulta por id — 22
8. Publicar el nuevo sitio — 24
9. Configurar reintentos y retención de logs — 27
10. Crear la alarma — 30
11. Etiquetar los recursos — 36
12. Verificación de punta a punta — 36
13. Costos — 47
14. Problemas frecuentes — 51
15. Limpieza — 51

1. Antes de empezar

Este manual explica, paso a paso, cómo hacer que una gaceta que publica noticias con imagen en AWS guarde cada noticia **en cuanto la imagen llega a S3**, y cómo vigilar ese proceso con logs, una métrica y una alarma de CloudWatch. Está escrito para un equipo que todavía no lo tiene: si sigues las secciones en orden, al final tendrás lo mismo que construimos nosotros y podrás comprobarlo con las pruebas de la sección «Verificación de punta a punta».

Hoy, en la mayoría de las gacetas del grupo, el navegador sube la imagen y después llama a una segunda función para guardar la noticia. Si esa segunda llamada falla, la imagen queda sin noticia y nadie se entera. Con este cambio, S3 avisa a una función nueva en cuanto termina de recibir la imagen, y esa función guarda la noticia (figura 1).

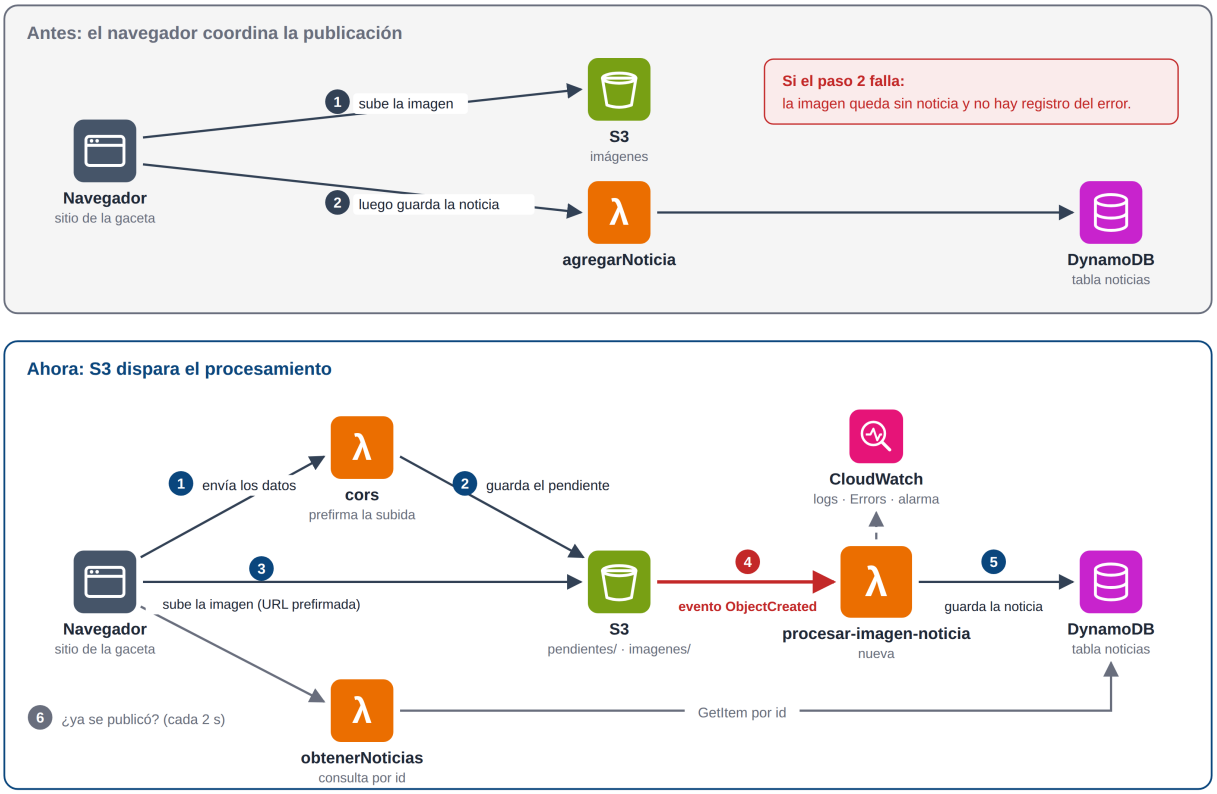


Figura 1. Arquitectura antes y después. La flecha roja es el evento de S3 que ahora inicia el guardado.

1.1. Cómo leer este manual

- Cada sección es un paso en la consola de AWS. Al final de cada una hay un recuadro verde «**Comprobación**» con lo que debes ver antes de continuar.
- El código está completo dentro del documento y es el mismo que está desplegado en nuestra cuenta (se inserta automáticamente desde la carpeta `codigo/` de los entregables al generar este PDF). Para copiarlo sin los números de línea, usa los archivos de esa carpeta.
- Las capturas son de nuestra cuenta (consola en español). El recuadro rojo señala lo que se configura o se comprueba.
- Tiempo estimado: unas dos horas, incluidas las pruebas.

1.2. Valores que debes reemplazar

En el código y en las capturas aparecen los nombres de nuestra gaceta. Antes de empezar, anota los tuyos y reemplázalos donde indica la tabla 1. Todos los recursos deben estar en la misma región.

Tabla 1. Valores de nuestra gaceta y dónde aparecen.

Dato	Valor en este manual	Dónde reemplazarlo
Región	`us-east-2` (Ohio)	`procesar-imagen.mjs` líneas 8, 9 y 51; `prefirmar-noticia.mjs` línea 6; `obtener-noticias.mjs` líneas 6, 7 y 19; la política IAM
Número de cuenta	`216728240051`	Política IAM, líneas 8, 14 y 33
Bucket de imágenes (privado)	`mis-noticias-daniel-imagenes-906-2026`	Variable `BUCKET` de las funciones; `obtener-noticias.mjs` línea 8; política IAM, líneas 20 y 26; eventos de prueba
Bucket del sitio	`906-unsis-daniel`	Solo para subir `index.html` y `js/main.js`
Tabla de DynamoDB	`noticias` (clave `id`, cadena)	Variable `TABLA`; `obtener-noticias.mjs` líneas 27 y 33; política IAM, línea 33
Función de URL prefirmadas	`cors`	Es la que ya tienes para subir imágenes; se reemplaza su código
Función de lectura	`obtenerNoticias`	Se reemplaza su código
URL de las funciones	`https://…lambda-url.us-east-2.on.aws/`	`main.js`, líneas 760 a 762

Los nombres nuevos (`procesar-imagen-noticia`, `procesar-imagen-noticia-rol` y `gaceta-procesadora-errores`) puedes conservarlos tal cual.

2. Revisar lo que ya existe

El cambio se apoya en recursos que la gaceta ya debe tener funcionando. Revísalos antes de tocar nada; si alguno falta, primero hay que crearlo.

Tabla 2. Requisitos previos.

Recurso	Qué debe cumplir
Sitio en S3	Publica `index.html` y `js/main.js` con el formulario «Crear noticia».
Bucket de imágenes	Privado y con CORS que permita `PUT` desde el sitio (figura 4).
Tabla `noticias`	Clave de partición `id` (String) y capacidad bajo demanda (figura 5).
Función de prefirmadas	Tiene URL de función con CORS: origen del sitio, encabezado `content-type`, métodos `POST` y `GET` (figura 3). Su rol permite `s3:PutObject` en el bucket de imágenes.
Función de lectura	Tiene URL de función. Su rol permite `dynamodb:Scan` y `dynamodb:GetItem` en la tabla y `s3:GetObject` en `imagenes/*`. **`GetItem` es nuevo**: agrégalo si no lo tiene.

En nuestra cuenta, las tres funciones originales usan el rol `PowerUserRole`, que ya incluía estos permisos.

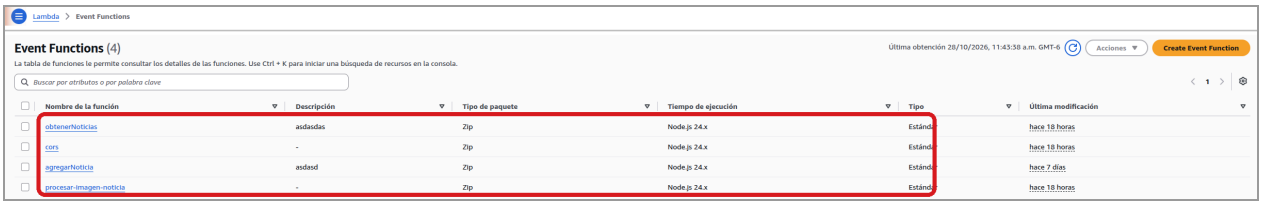


Figura 2. Lambda › Funciones. Las funciones existentes (`obtenerNoticias`, `cors` y `agregarNoticia`) más la procesadora que crearemos. Las cuatro usan Node.js 24.x.

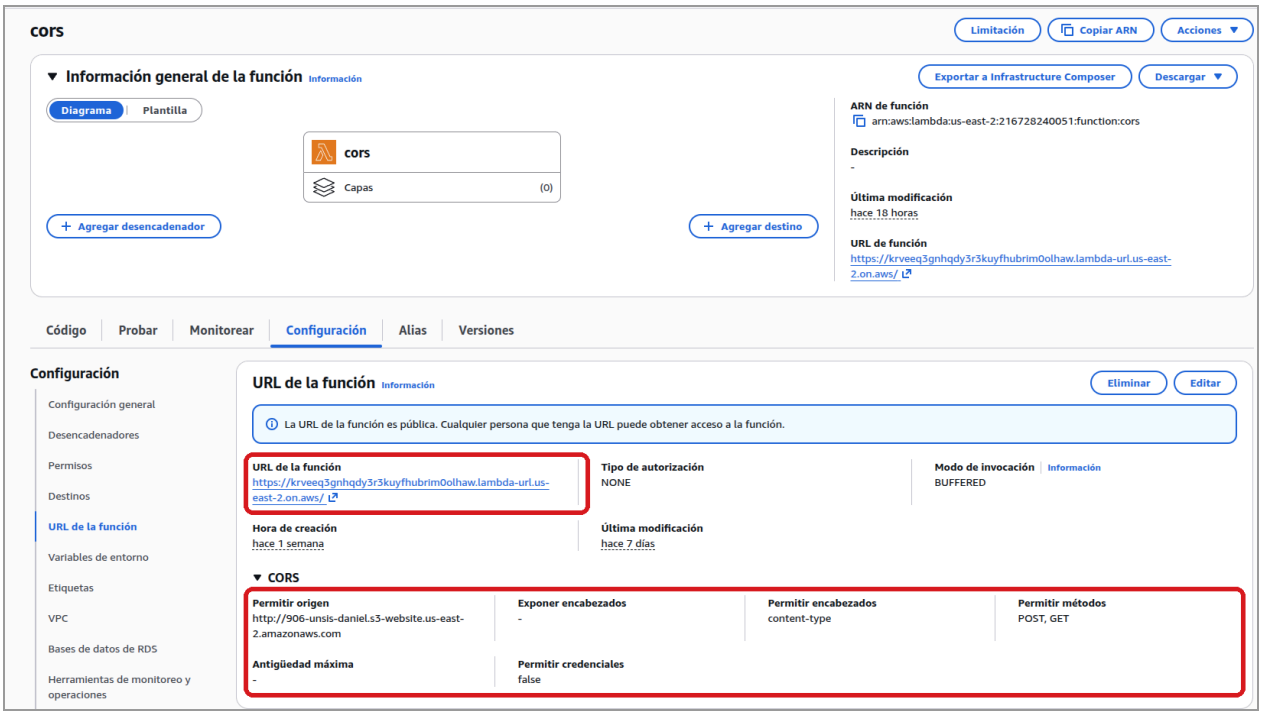


Figura 3. Lambda › cors › Configuración › URL de la función. CORS con el origen del sitio, el encabezado `content-type` y los métodos `POST` y `GET`.

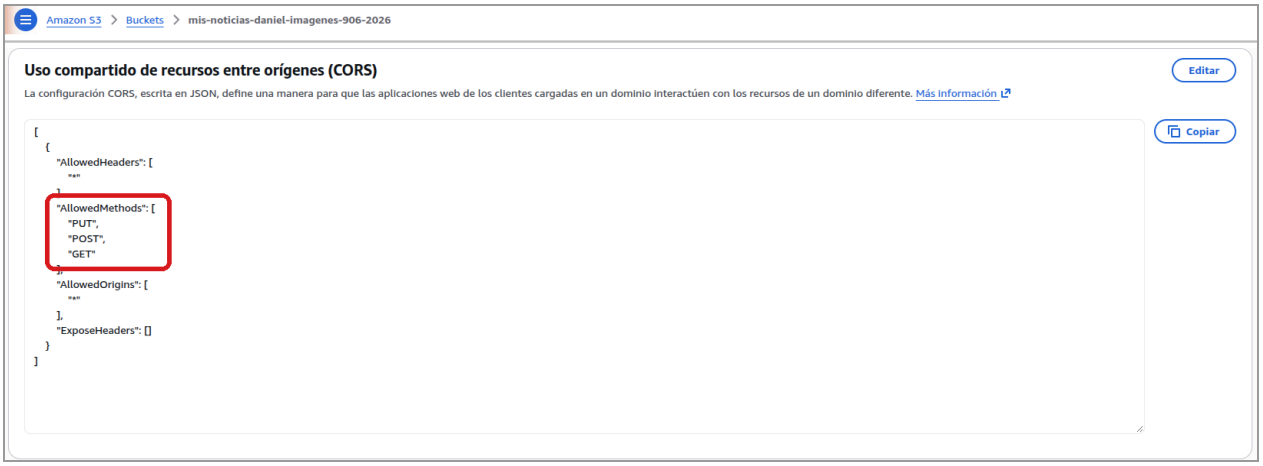


Figura 4. S3 › bucket de imágenes › Permisos › CORS. Debe permitir `PUT`, que es como el navegador sube la imagen.

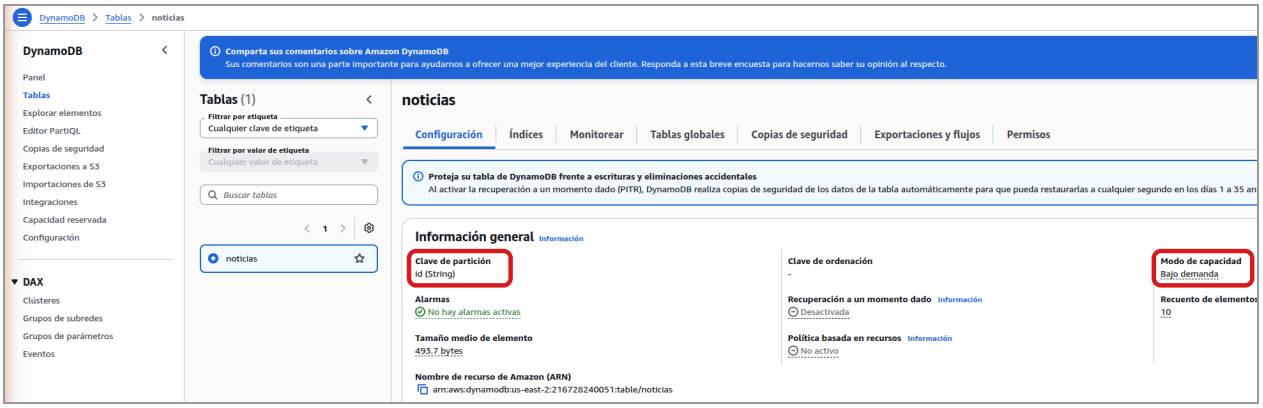


Figura 5. DynamoDB › Tablas › noticias. Clave de partición `id (String)` y modo de capacidad bajo demanda.

3. Actualizar la función de prefirmadas

Ahora la web enviará primero los datos de la noticia. Esta función los valida, los guarda como «pendiente» en `pendientes/ID.json` y devuelve la URL para subir la imagen a `imagenes/ID/foto.ext`. Conserva el modo anterior (`POST ?file=nombre`) para no romper a quien tenga la página vieja en caché.

1. Abre **Lambda › Funciones › cors › Código**. Copia el contenido de `index.mjs` a un archivo de respaldo (nosotros lo guardamos como `prefirmadas-original.mjs`).
2. Borra el contenido de `index.mjs`, pega el código siguiente y pulsa **Deploy**.

prefirmar-noticia.mjs — se pega en index.mjs de la función cors

```
1 import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';
2 import { getSignedUrl } from '@aws-sdk/s3-request-presigner';
```

```
import { randomUUID } from 'node:crypto';

const BUCKET = process.env.BUCKET || 'mis-noticias-daniel-imagenes-906-2026';
const s3 = new S3Client({ region: 'us-east-2' });
const EXTENSIONES = { 'image/jpeg': 'jpg', 'image/png': 'png', 'image/webp': 'webp' };
const UUID = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
const responder = (statusCode, datos) => ({ statusCode, headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(datos) });
const log = (evento, datos = {}) => console.log(JSON.stringify({ evento, funcion: 'prefirmar', ...datos }));

// Valida los mismos campos que la Lambda de agregar para no dejar pendientes inservibles.
function validar(body) {
  const { titulo, detalle, fecha, categoria = 'General', alt = '', tipo } = body;
  if ([titulo, detalle, fecha, categoria].some(x => typeof x !== 'string' || !x.trim())) return 'Faltan título, detalle, fecha o categoría.';
  if (titulo.length > 180 || detalle.length > 30000 || categoria.length > 60 || !/^\d{4}-\d{2}-\d{2}$/.test(fecha)) return 'Revisa la longitud de los campos y el formato de la fecha.';
  if (typeof alt !== 'string' || !alt.trim() || alt.length > 300) return 'La imagen necesita una descripción de máximo 300 caracteres.';
  if (!EXTENSIONES[tipo]) return 'La imagen debe ser JPG, PNG o WebP.';
  if (body.id !== undefined && (typeof body.id !== 'string' || !UUID.test(body.id))) return 'Identificador no válido.';
  return '';
}

export const handler = async (event, context) => {
  const inicio = Date.now();
  const metodo = event.requestContext?.http?.method || event.httpMethod;
  if (metodo === 'OPTIONS') return responder(200, {});
  if (metodo && metodo !== 'POST') return responder(405, { error: 'Utiliza POST.' });
  let body = {};
  try {
    if (event.body) body = JSON.parse(event.isBase64Encoded ? Buffer.from(event.body, 'base64').toString('utf8') : event.body);
  } catch { return responder(400, { error: 'JSON no válido.' }); }

  try {
    // Flujo anterior (POST ?file=nombre sin cuerpo): se conserva mientras se migra el frontend.
    if (!body || typeof body !== 'object' || !body.titulo) {
      const nombre = String(event.queryStringParameters?.file || '').trim();
      const extension = nombre.split('.').pop().toLowerCase();
      const tipo = { jpg: 'image/jpeg', jpeg: 'image/jpeg', png: 'image/png', webp: 'image/webp' }[extension];
      if (!nombre || !tipo) return responder(400, { error: 'Archivo no válido' });
      const key = `imagenes/${randomUUID()}-${nombre.replace(/[^a-zA-Z0-9._-]/g, '_')}`;
      const uploadUrl = await getSignedUrl(s3, new PutObjectCommand({ Bucket: BUCKET, Key: key, ContentType: tipo }), { expiresIn: 300 });
      log('url_legada', { requestId: context?.awsRequestId, key, duracionMs: Date.now() - inicio });
      return responder(200, { uploadUrl, key });
    }

    const error = validar(body);
    if (error) {
      log('validacion_rechazada', { requestId: context?.awsRequestId, motivo: error });
      return responder(400, { error });
    }
    const id = body.id || randomUUID();
    const key = `imagenes/${id}/foto.${EXTENSIONES[body.tipo]}`;
    const pendiente = {
      id, titulo: body.titulo.trim(), detalle: body.detalle.trim(), fecha: body.fecha,
      categoria: (body.categoria || 'General').trim(), alt: body.alt.trim(), imagenKey: key,
      solicitadoEn: new Date().toISOString()
    };
    // El JSON se guarda antes de emitir la URL: la procesadora siempre lo encontrará al llegar la imagen.
    await s3.send(new PutObjectCommand({
```

```
60        Bucket: BUCKET, Key: `pendientes/${id}.json`, Body: JSON.stringify(pendiente), ContentType: 'applicatio
   n/json'
61      }));
62      const uploadUrl = await getSignedUrl(s3, new PutObjectCommand({ Bucket: BUCKET, Key: key, ContentType: bod
   y.tipo }), { expiresIn: 300 });
63      log('pendiente_creado', { noticiaId: id, requestId: context?.awsRequestId, bucket: BUCKET, key, duracionM
   s: Date.now() - inicio });
64      return responder(200, { uploadUrl, noticiaId: id, key });
65    } catch (error) {
66      console.error(JSON.stringify({ evento: 'fallo', funcion: 'prefirmar', requestId: context?.awsRequestId, er
   ror: error.name, mensaje: error.message }));
67      return responder(500, { error: 'No se pudo preparar la subida.' });
68    }
69  };
```

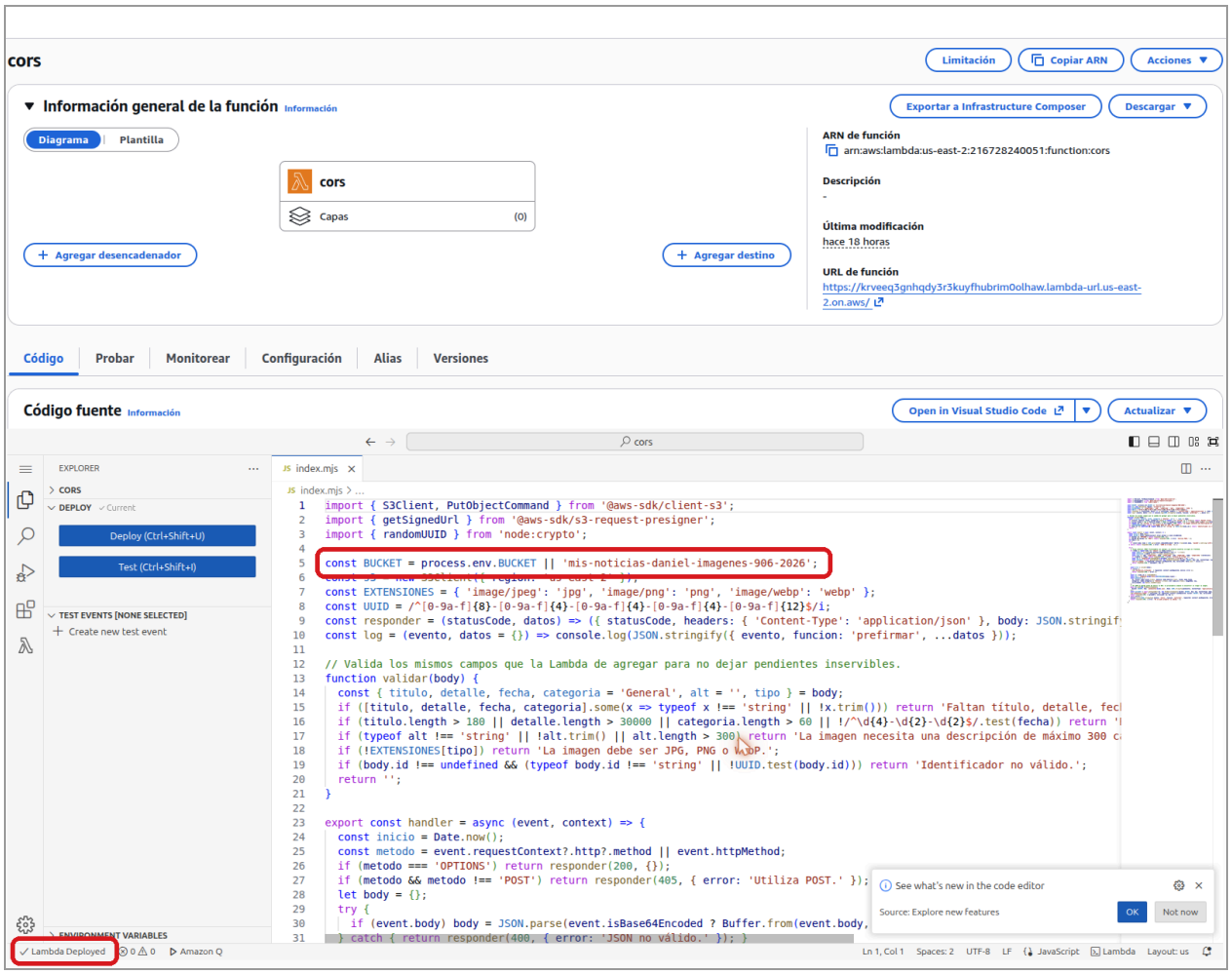


Figura 6. Código pegado en `index.mjs`: la línea 5 lleva el nombre del bucket y el pie del editor indica **Lambda Deployed**.

Comprobación. En la pestaña **Probar**, crea un evento con este JSON y pulsa **Probar**:

```
{
  "requestContext": { "http": { "method": "POST" } },
  "body": "{\"titulo\":\"Prueba\",\"detalle\":\"Texto de prueba\",\"fecha\":\"2026-11-04\",\"categori
a\":\"General\",\"alt\":\"Foto de prueba\",\"tipo\":\"image/jpeg\"}"
}
```

La respuesta debe tener `"statusCode": 200` y un `body` con `uploadUrl`, `noticiaId` y `key` de la forma `imagenes/ID/foto.jpg`. En el bucket de imágenes aparece `pendientes/ID.json`; puedes borrarlo. Si cambias la fecha por `"04/11/2026"`, la respuesta debe ser `400`.

4. Crear el rol de la procesadora

La función nueva solo necesita escribir sus propios logs, leer los pendientes y guardar en la tabla. Le creamos un rol con exactamente eso (mínimo privilegio), antes de crear la función para poder elegirlo desde el principio.

1. En **IAM › Roles › Crear rol**, elige **Servicio de AWS** y el caso de uso **Lambda** (figura 7).
2. En **Agregar permisos** no marques ninguna política; pulsa **Siguiente**.
3. Nombre: `procesar-imagen-noticia-rol`. Agrega las etiquetas `proyecto`, `equipo` y `entorno`, y pulsa **Crear rol** (figura 8). La descripción no acepta el carácter `>`.
4. Abre el rol y entra a **Agregar permisos › Crear política insertada › JSON**. Pega la política siguiente, con tu región, cuenta, bucket y tabla.
5. Pulsa **Siguiente**, nómbrala `ProcesadoraMinimoPrivilegio` y pulsa **Crear política**.

procesadora-politica.json — política insertada ProcesadoraMinimoPrivilegio

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CrearGrupoDeLogs",
      "Effect": "Allow",
      "Action": "logs:CreateLogGroup",
      "Resource": "arn:aws:logs:us-east-2:216728240051:*"
    },
    {
      "Sid": "EscribirLogsDeLaProcesadora",
      "Effect": "Allow",
      "Action": ["logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "arn:aws:logs:us-east-2:216728240051:log-group:/aws/lambda/procesar-imagen-noticia:*"
    },
    {
      "Sid": "LeerPendientes",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mis-noticias-daniel-imagenes-906-2026/pendientes/*"
```

```
    },
    {
      "Sid": "DistinguirPendienteAusente",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::mis-noticias-daniel-imagenes-906-2026",
      "Condition": { "StringLike": { "s3:prefix": "pendientes/*" } }
    },
    {
      "Sid": "GuardarNoticia",
      "Effect": "Allow",
      "Action": "dynamodb:PutItem",
      "Resource": "arn:aws:dynamodb:us-east-2:216728240051:table/noticias"
    }
  ]
}
```

Tabla 3. Para qué sirve cada permiso de la política.

Permiso	Para qué
`logs:CreateLogGroup`, `CreateLogStream`, `PutLogEvents`	Escribir solo en su grupo de logs `/aws/lambda/procesar-imagen-noticia`.
`s3:GetObject` en `pendientes/*`	Leer el JSON de la noticia. No puede leer las imágenes ni el resto del bucket.
`s3:ListBucket` con prefijo `pendientes/*`	Que un pendiente inexistente responda `NoSuchKey` y no `AccessDenied`; así la función distingue «no hay datos» de «no tengo permiso».
`dynamodb:PutItem` en la tabla	Guardar la noticia. No puede leer, borrar ni recorrer la tabla.

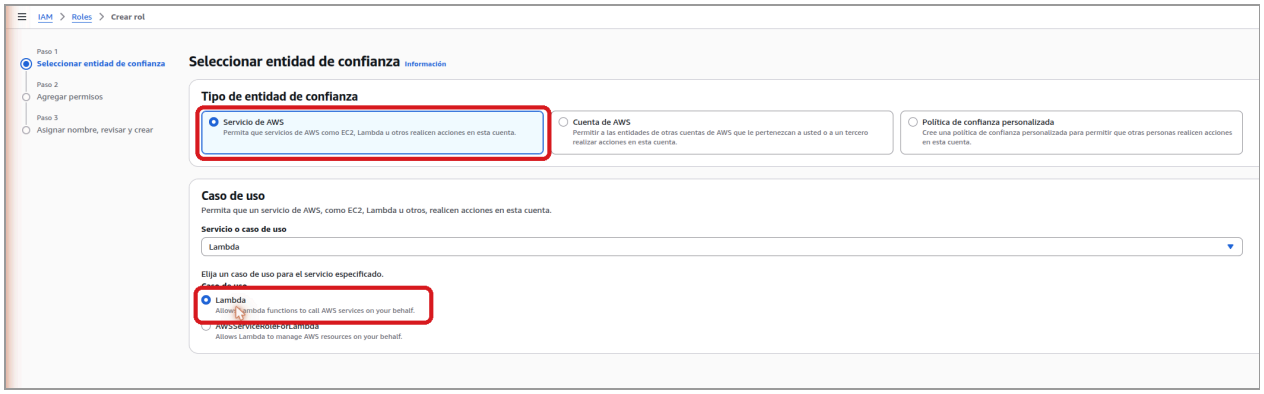


Figura 7. IAM › Roles › Crear rol, paso 1. Servicio de AWS y caso de uso Lambda.

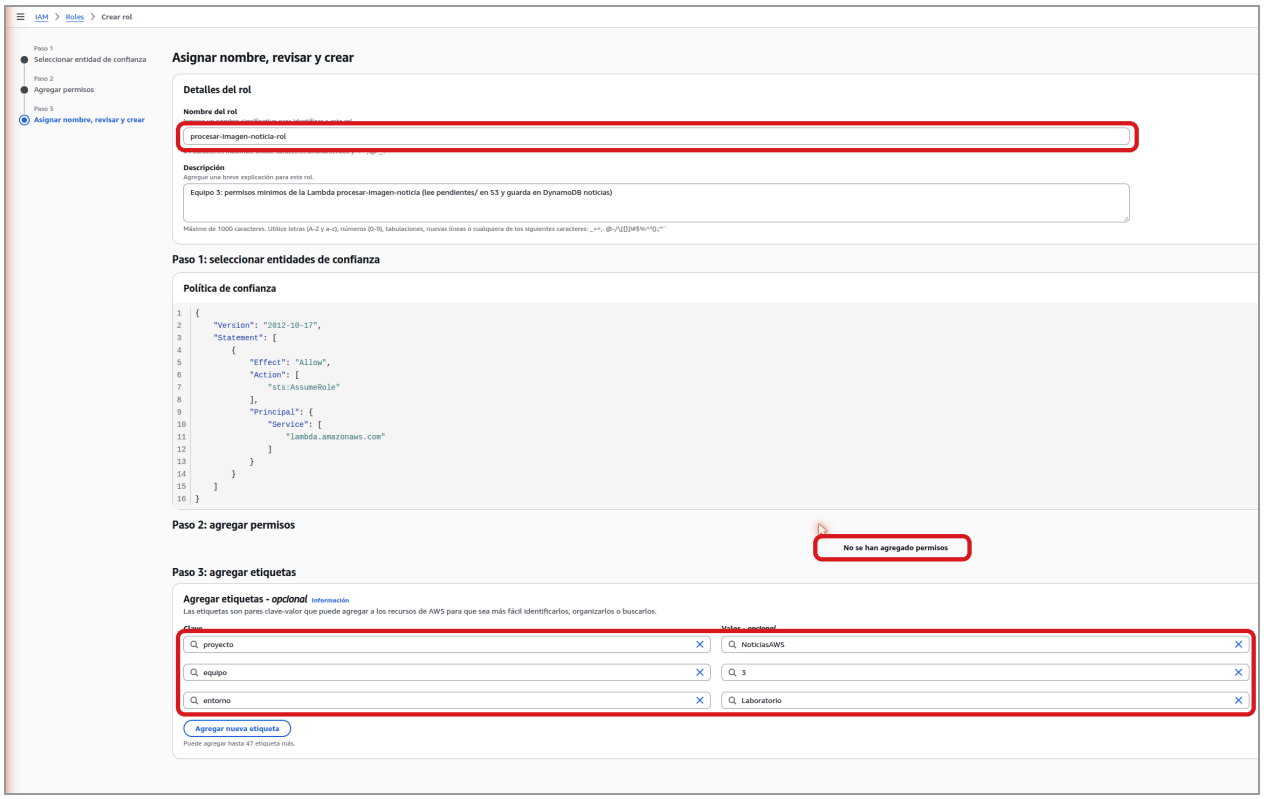


Figura 8. Paso 3. Nombre del rol, ninguna política administrada («No se han agregado permisos») y las tres etiquetas.

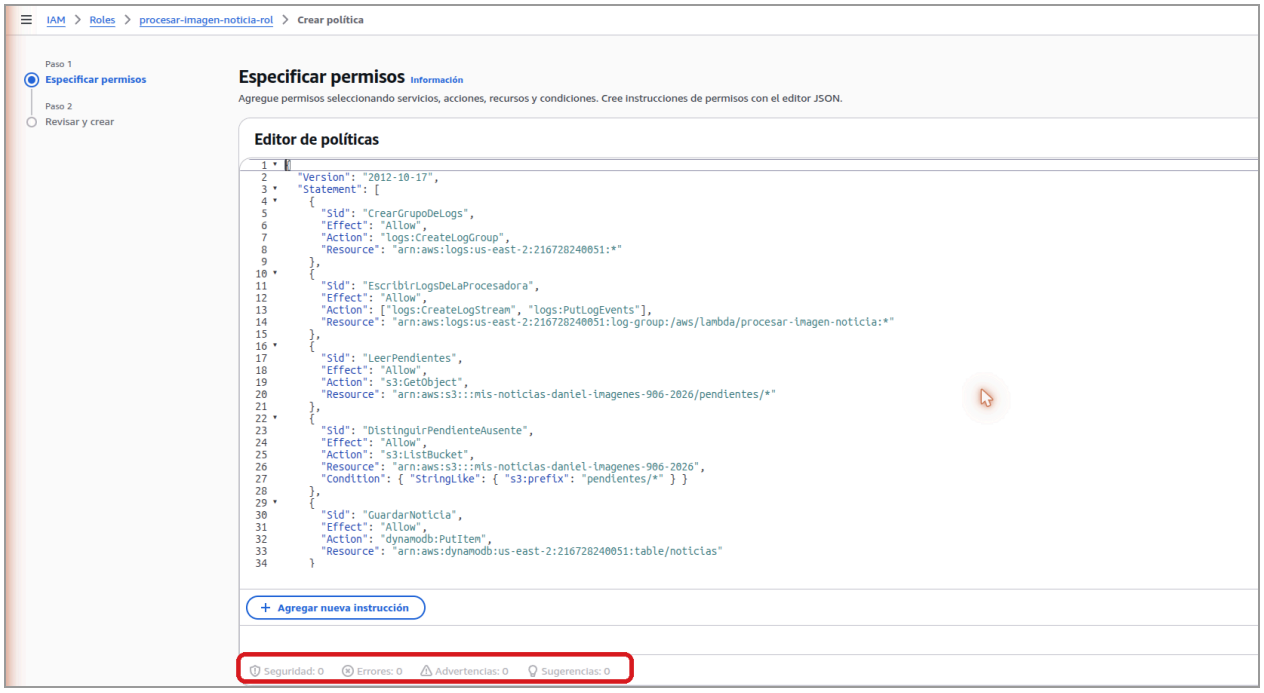


Figura 9. Política pegada en el editor JSON. Abajo, el validador marca 0 en seguridad, errores, advertencias y sugerencias.

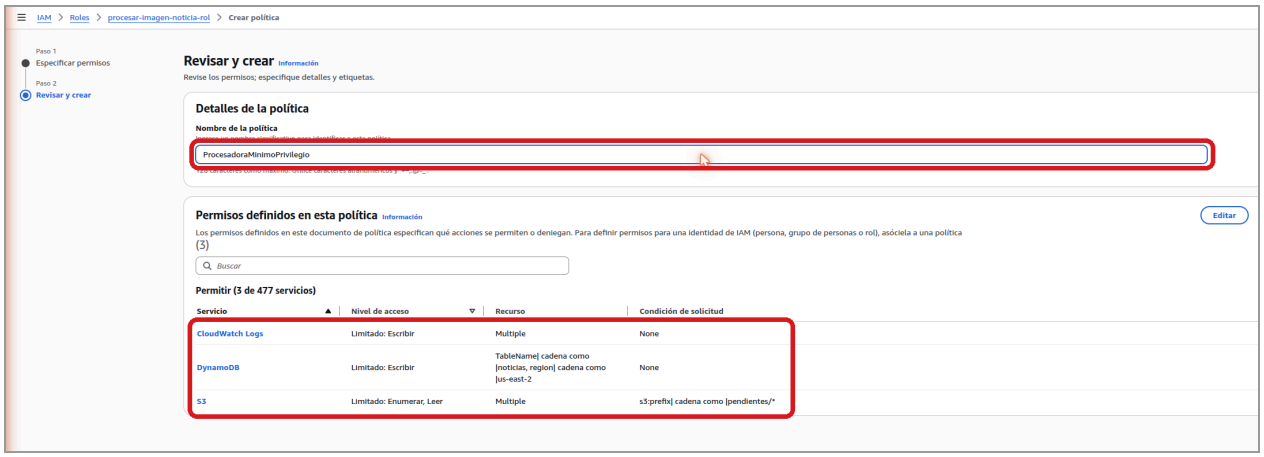


Figura 10. Revisar y crear. Nombre `ProcesadoraMinimoPrivilegio` y resumen: escritura limitada en CloudWatch Logs y DynamoDB, y lectura limitada en S3 con la condición de prefijo `pendientes/*`.

Comprobación. El rol muestra **Políticas de permisos (1)** con `ProcesadoraMinimoPrivilegio` de tipo «Cliente insertado» (figura 11).

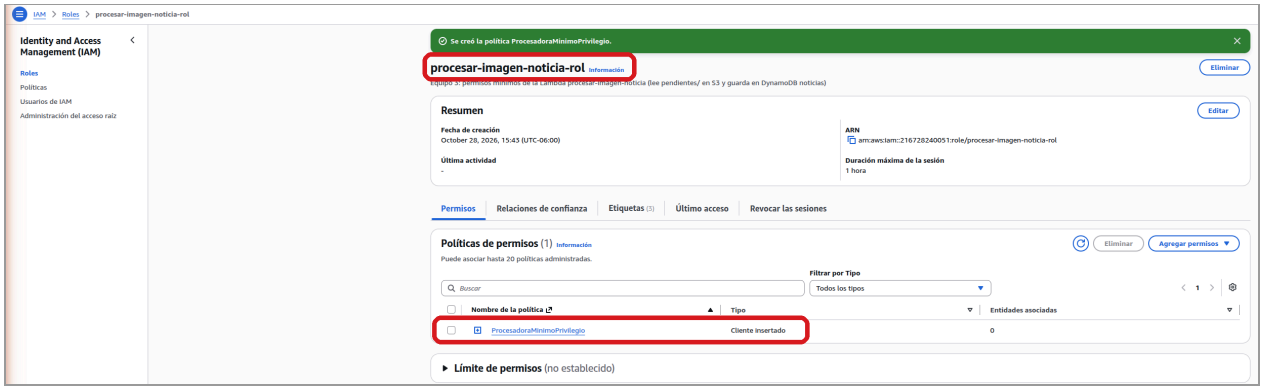


Figura 11. Rol creado con la política insertada como único permiso.

5. Crear la función procesadora

1. En **Lambda › Crear función › Crear desde cero**, escribe el nombre `procesar-imagen-noticia` y elige **Node.js 24.x**.
2. En **Configuración adicional**, activa **Rol de ejecución personalizada** y elige `procesar-imagen-noticia-rol`. En la figura 12 aparece `PowerUserRole` porque nosotros cambiamos el rol después; tú puedes elegir el nuevo desde aquí.
3. Pulsa **Crear función**. En **Código**, reemplaza el contenido de `index.mjs` por el código siguiente y pulsa **Deploy**.

4. En **Configuración › Configuración general › Editar**, sube el tiempo de espera a **30 segundos** y confirma el rol (figura 14).
5. En **Configuración › Variables de entorno › Editar**, agrega `BUCKET` con el nombre de tu bucket de imágenes y `TABLA` con `noticias` (figura 15).

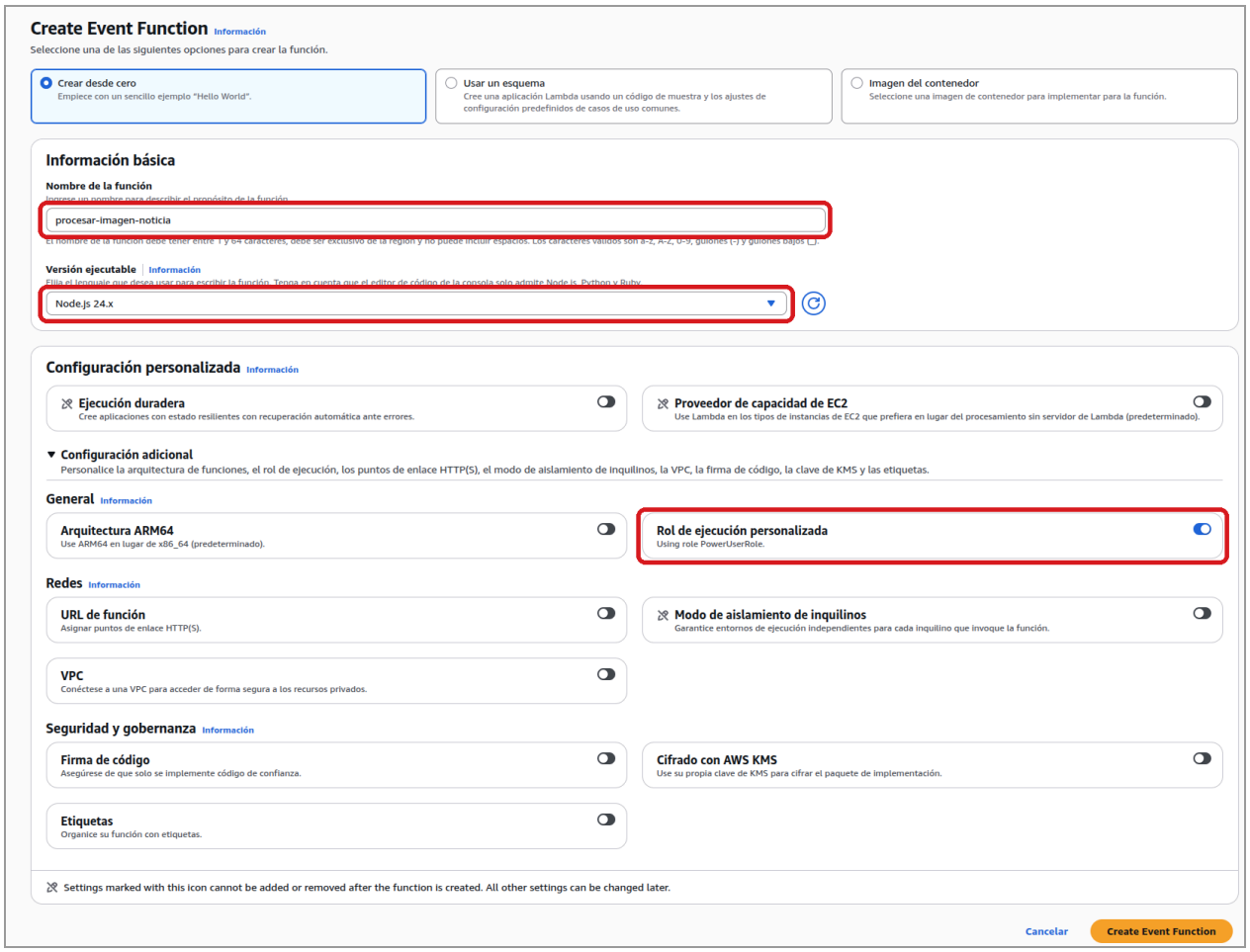


Figura 12. Lambda › Crear función. Nombre, versión ejecutable y rol de ejecución personalizado.

Así trabaja la función: toma la clave de la imagen, obtiene el id, lee `pendientes/ID.json`, valida los campos y guarda la noticia con la condición `attribute_not_exists(id)`, que impide duplicarla. Los errores de datos (pendiente ausente, campos inválidos) se registran y no se reintentan; los errores de infraestructura (por ejemplo, DynamoDB no responde) se relanzan para que Lambda reintente y CloudWatch cuente el error.

procesar-imagen.mjs — se pega en index.mjs de procesar-imagen-noticia

```
import { DynamoDBClient } from '@aws-sdk/client-dynamodb';
import { DynamoDBDocumentClient, PutCommand } from '@aws-sdk/lib-dynamodb';
import { S3Client, GetObjectCommand } from '@aws-sdk/client-s3';

// TABLA permite apuntar a una tabla inexistente para simular fallos sin tocar la tabla compartida (P04).
```

```
const TABLA = process.env.TABLA || 'noticias';
const BUCKET = process.env.BUCKET || 'mis-noticias-daniel-imagenes-906-2026';
const db = DynamoDBDocumentClient.from(new DynamoDBClient({ region: 'us-east-2' }));
const s3 = new S3Client({ region: 'us-east-2' });
const CLAVE = /^imagenes\/([0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12})\/foto\.(jpg|png|webp)$/i;
const log = (evento, datos = {}) => console.log(JSON.stringify({ evento, funcion: 'procesar-imagen', ...datos }));

class ErrorTerminal extends Error {}

function validar(p, id, key) {
  if (!p || typeof p !== 'object') throw new ErrorTerminal('El pendiente no es un objeto JSON.');
  if (p.id !== id || p.imagenKey !== key) throw new ErrorTerminal('El pendiente no corresponde a la imagen.');
  if ([p.titulo, p.detalle, p.fecha, p.categoria, p.alt].some(x => typeof x !== 'string' || !x.trim())) throw new ErrorTerminal('Faltan campos en el pendiente.');
  if (p.titulo.length > 180 || p.detalle.length > 30000 || p.categoria.length > 60 || p.alt.length > 300 || !/^\d{4}-\d{2}-\d{2}$/.test(p.fecha)) throw new ErrorTerminal('Campos fuera de límite.');
}

async function procesar(record, requestId) {
  const inicio = Date.now();
  const bucket = record.s3?.bucket?.name;
  // Las claves de S3 llegan codificadas como URL y con + en lugar de espacios.
  const key = decodeURIComponent(String(record.s3?.object?.key || '').replace(/\+/g, ' '));
  const base = { requestId, bucket, key };
  log('inicio', base);
  if (bucket !== BUCKET) throw new ErrorTerminal(`Bucket no esperado: ${bucket}`);
  const coincidencia = key.match(CLAVE);
  // Imágenes del flujo anterior (imagenes/<marca>-<nombre>) no tienen pendiente: se ignoran sin error.
  if (!coincidencia) { log('omitido_sin_formato', base); return; }
  const noticiaId = coincidencia[1].toLowerCase();
  const tamano = record.s3.object.size;
  if (!tamano || tamano > 10 * 1024 * 1024) throw new ErrorTerminal(`Tamaño no válido: ${tamano}`);

  let pendiente;
  try {
    const objeto = await s3.send(new GetObjectCommand({ Bucket: bucket, Key: `pendientes/${noticiaId}.json` }));
    pendiente = JSON.parse(await objeto.Body.transformToString());
  } catch (error) {
    if (error.name === 'NoSuchKey' || error instanceof SyntaxError) throw new ErrorTerminal(`Pendiente ausente o inválido (${error.name}).`);
    throw error;
  }
  validar(pendiente, noticiaId, key);
  log('validado', { ...base, noticiaId });

  const item = {
    id: noticiaId, titulo: pendiente.titulo.trim(), detalle: pendiente.detalle.trim(), fecha: pendiente.fecha,
    categoria: pendiente.categoria.trim(), alt: pendiente.alt.trim(),
    imagen: `https://${bucket}.s3.us-east-2.amazonaws.com/${key}`,
    origen: 's3-evento', creadoEn: new Date().toISOString()
  };
  try {
    await db.send(new PutCommand({ TableName: TABLA, Item: item, ConditionExpression: 'attribute_not_exists(id)' }));
    log('guardado', { ...base, noticiaId, tabla: TABLA, duracionMs: Date.now() - inicio });
  } catch (error) {
    // Un reintento o un evento repetido no crea un segundo registro (P07).
    if (error.name === 'ConditionalCheckFailedException') { log('duplicado', { ...base, noticiaId, duracionMs: Date.now() - inicio }); return; }
    throw error;
  }
}
```

```
63
64 export const handler = async (event, context) => {
65   const requestId = context?.awsRequestId;
66   const fallos = [];
67   for (const record of event.Records || []) {
68     try {
69       await procesar(record, requestId);
70     } catch (error) {
71       const key = record.s3?.object?.key;
72       if (error instanceof ErrorTerminal) {
73         // Validación: reintentar no lo corrige. Se registra y no se relanza.
74         log('validacion_fallida', { requestId, key, motivo: error.message });
75       } else {
76         console.error(JSON.stringify({ evento: 'fallo', funcion: 'procesar-imagen', requestId, key, error: err
   or.name, mensaje: error.message }));
77         fallos.push(error);
78       }
79     }
80   }
81   // Relanzar los fallos operativos incrementa la métrica Errors y activa los reintentos asíncronos (P04-P06).
82   if (fallos.length) throw new Error(`${fallos.length} registro(s) fallaron: ${fallos.map(e => e.name).join(',
   ')}`);
83   return { procesados: (event.Records || []).length };
84 };
```

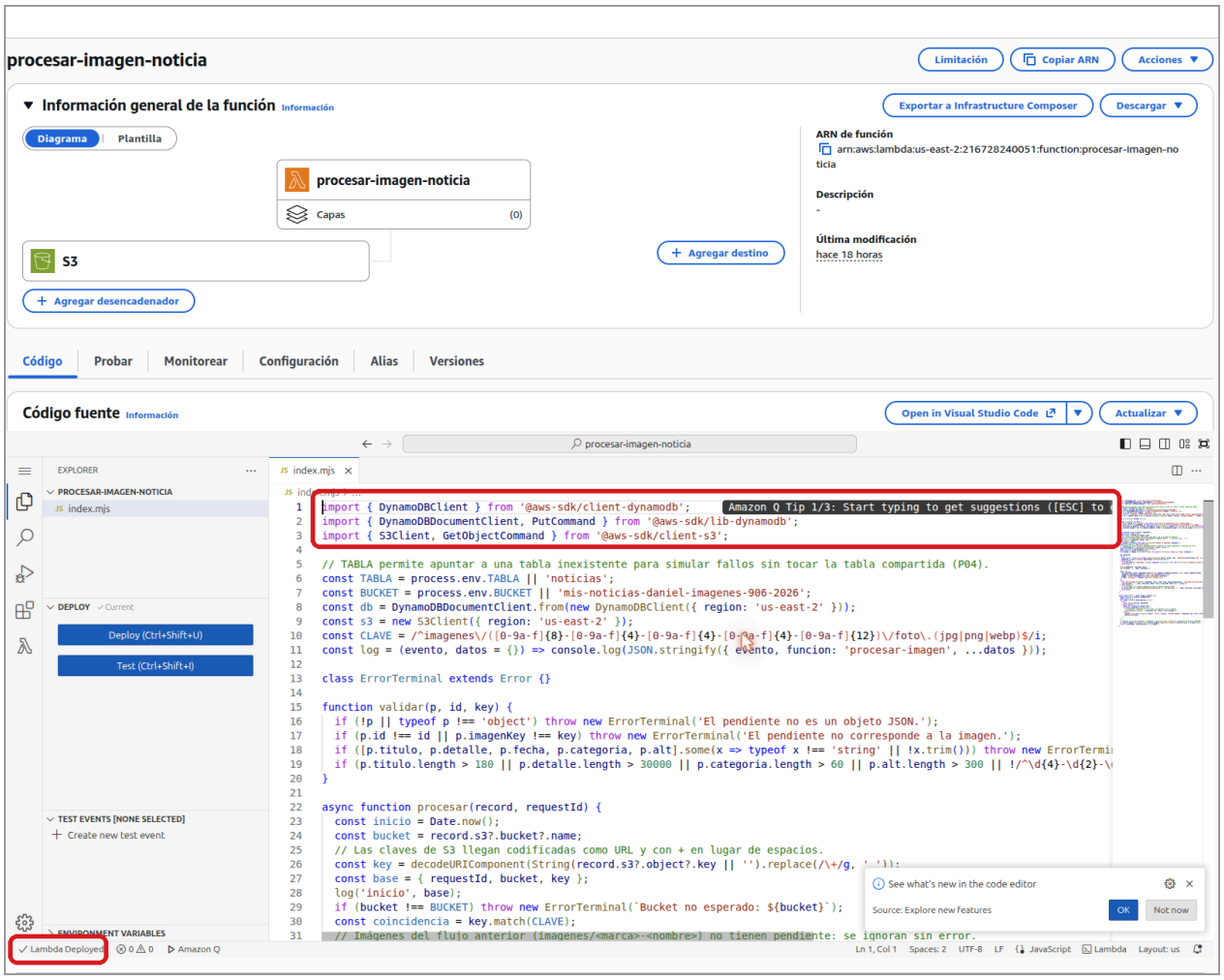


Figura 13. Código desplegado en la procesadora. Arriba ya se ve el disparador de S3, que se agrega en la sección siguiente.

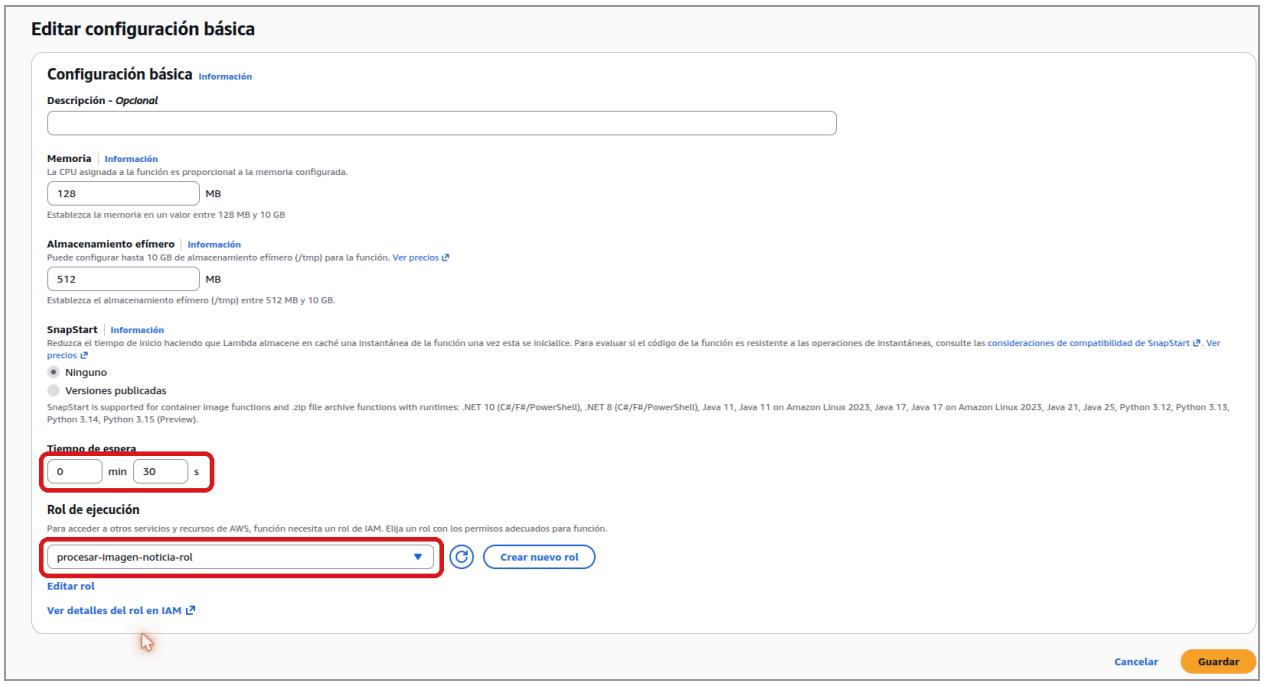


Figura 14. Configuración general › Editar. Tiempo de espera de 30 s y el rol `procesar-imagen-noticia-rol`.

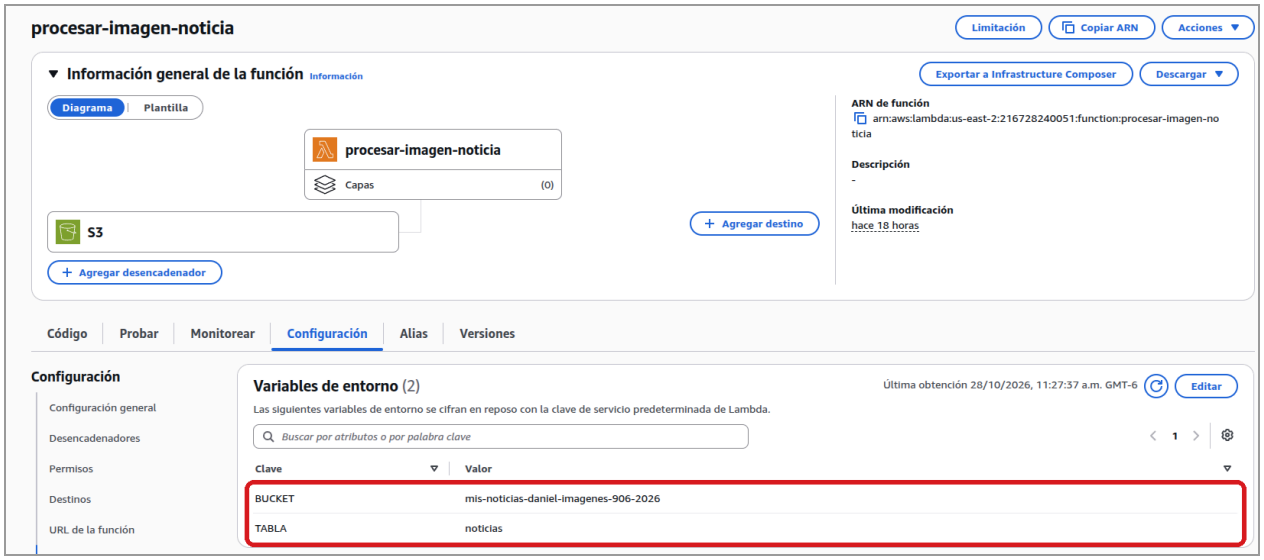


Figura 15. Variables de entorno. `TABLA` permite simular un fallo de DynamoDB sin tocar la tabla real (prueba P04).

Comprobación. En **Probar**, pega el evento siguiente (cambia el bucket por el tuyo) y pulsa **Probar**. Como ese id no tiene pendiente, la ejecución debe terminar **sin errores**, con la respuesta `{"procesados": 1}` y, en el log, `"evento":"validacion_fallida"` con el motivo `NoSuchKey`. Esta primera ejecución también crea el grupo de logs que se configura más adelante.

pruebas/evento-s3-sin-pendiente.json

```
{
  "Records": [{
    "eventSource": "aws:s3",
    "awsRegion": "us-east-2",
    "eventName": "ObjectCreated:Put",
    "s3": {
      "bucket": { "name": "mis-noticias-daniel-imagenes-906-2026" },
      "object": { "key": "imagenes/00000000-0000-4000-8000-000000000000/foto.jpg", "size": 123456 }
    }
  }]
}
```

Si el log dice `AccessDenied` en lugar de `NoSuchKey`, falta el permiso `s3:ListBucket` de la política.

6. Conectar S3 con la función

1. En la procesadora, pulsa **Agregar desencadenador** y elige **S3**.
2. Bucket: tu bucket de imágenes. Tipo de evento: **Todos los eventos de creación de objetos**. Prefijo: `imagenes/`. Sufijo: vacío.
3. Marca el aviso de **invocación recurrente** y pulsa **Agregar**. No hay ciclo: la función no escribe en S3, y los pendientes están fuera de `imagenes/`.

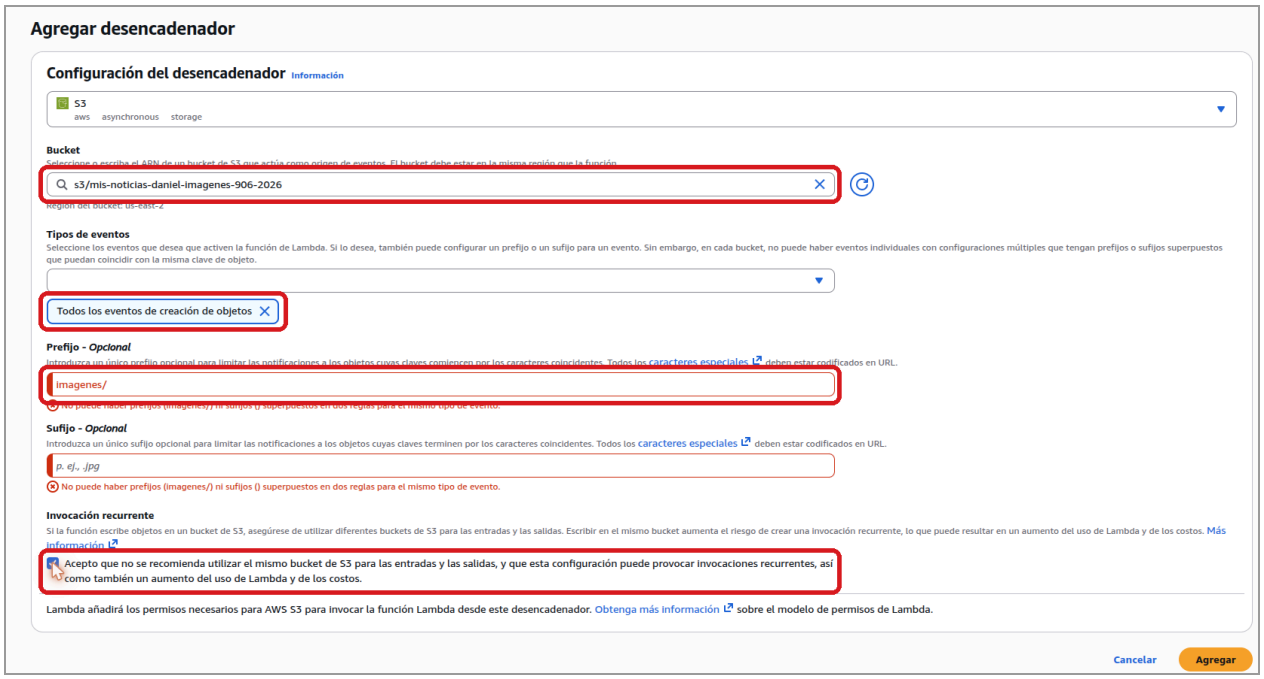


Figura 16. Agregar desencadenador. Bucket, tipo de evento, prefijo y aviso de invocación recurrente. El mensaje rojo aparece en nuestra captura porque la regla ya existía cuando la tomamos; en una cuenta nueva no sale.

Lambda agrega por sí sola el permiso para que S3 invoque la función, limitado a tu cuenta y a tu bucket (figura 18), y la regla aparece también en las propiedades del bucket (figura 19).

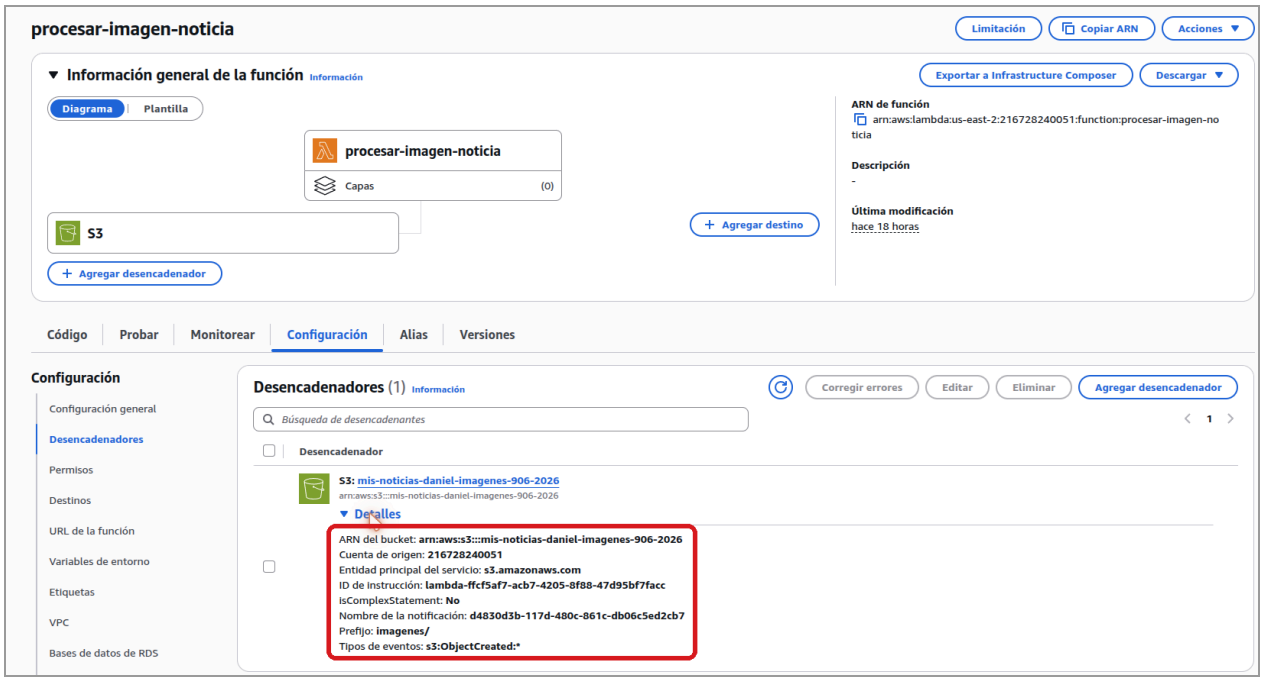


Figura 17. Configuración › Desencadenadores. Prefijo `imagenes/` y tipo de evento `s3:ObjectCreated:*`.

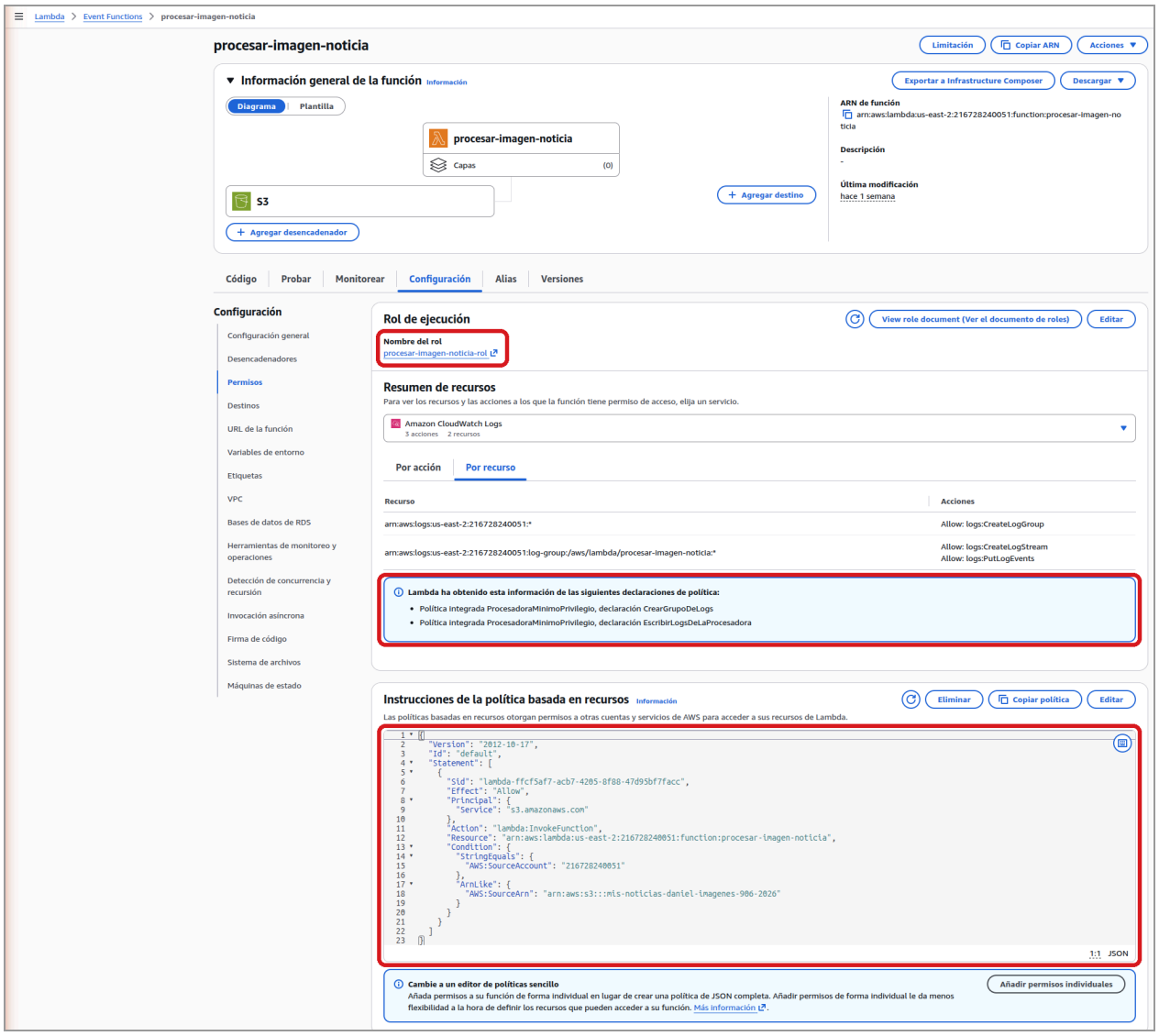


Figura 18. Configuración › Permisos (captura del 4 de noviembre). Arriba, el rol `procesar-imagen-noticia-rol` y las declaraciones de `ProcesadoraMinimoPrivilegio`. Abajo, la política basada en recursos que agregó Lambda: permite a `s3.amazonaws.com` invocar la función solo desde tu cuenta (`AWS:SourceAccount`) y tu bucket (`AWS:SourceArn`).



Figura 19. S3 › bucket › Propiedades › Notificaciones de eventos. Filtro `imagenes/` y destino `procesar-imagen-noticia`.

Comprobación. El diagrama de la función muestra **S3** a la izquierda, y el bucket tiene una notificación de eventos con destino en la procesadora. Si al guardar aparece «No puede haber prefijos superpuestos», ya hay otra regla sobre `imagenes/` en el bucket: revísala en **Propiedades › Notificaciones de eventos**.

7. Agregar la consulta por id

Como el guardado ahora es asíncrono, la web necesita preguntar si la noticia ya se publicó. Reemplazamos el código de `obtenerNoticias` para que, además del listado completo, responda `GET ?id=ID` con `{"estado":"pendiente"}` o `{"estado":"publicada","item":…}`. La lectura por id es consistente, así que en cuanto la procesadora guarda, la consulta la ve.

1. Abre **Lambda › obtenerNoticias › Código**, respalda el código actual y reemplázalo por el siguiente.
2. Pulsa **Deploy**. Recuerda que su rol necesita `dynamodb:GetItem`.

obtener-noticias.mjs — se pega en index.mjs de obtenerNoticias

```
import { DynamoDBClient } from '@aws-sdk/client-dynamodb';
import { DynamoDBDocumentClient, ScanCommand, GetCommand } from '@aws-sdk/lib-dynamodb';
import { S3Client, GetObjectCommand } from '@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

const db = DynamoDBDocumentClient.from(new DynamoDBClient({ region: 'us-east-2' }));
const s3 = new S3Client({ region: 'us-east-2' });
const BUCKET = 'mis-noticias-daniel-imagenes-906-2026';
const headers = { 'Content-Type': 'application/json' };
export const handler = async event => {
  const metodo = event.requestContext?.http?.method || event.httpMethod;
  if (metodo === 'OPTIONS') return { statusCode: 200, headers, body: '' };
  if (metodo && metodo !== 'GET') return { statusCode: 405, headers, body: JSON.stringify({ error: 'Utiliza GE
T.' }) };
  // Entrega URLs de lectura temporales para mostrar imágenes incluso con bucket privado.
  const firmar = async noticia => {
    if (!noticia.imagen) return noticia;
    let url;
    try { url = new URL(noticia.imagen); } catch { return noticia; }
    if (url.hostname !== `${BUCKET}.s3.us-east-2.amazonaws.com` || !url.pathname.startsWith('/imagenes/')) ret
urn noticia;
    const imagen = await getSignedUrl(s3, new GetObjectCommand({ Bucket: BUCKET, Key: decodeURIComponent(url.p
athname.slice(1)) }), { expiresIn: 3600 });
    return { ...noticia, imagen };
  };
  try {
    // ?id= consulta una sola noticia para seguir el procesamiento asíncrono sin recorrer la tabla.
    const id = event.queryStringParameters?.id;
    if (id) {
      const { Item } = await db.send(new GetCommand({ TableName: 'noticias', Key: { id }, ConsistentRead: true
}));
      return { statusCode: 200, headers, body: JSON.stringify(Item ? { estado: 'publicada', item: await firmar
(Item) } : { estado: 'pendiente', id }) };
    }
    const noticias = [];
    let cursor;
```

```
32      do {
33        const pagina = await db.send(new ScanCommand({ TableName: 'noticias', ExclusiveStartKey: cursor }));
34        noticias.push(...(pagina.Items || []));
35        cursor = pagina.LastEvaluatedKey;
36      } while (cursor);
37      const resultado = await Promise.all(noticias.map(firmar));
38      return { statusCode: 200, headers, body: JSON.stringify(resultado) };
39    } catch (error) {
40      console.error(error);
41      return { statusCode: 500, headers, body: JSON.stringify({ error: 'No se pudieron obtener las noticias.' })
    };
42    }
43  };
```

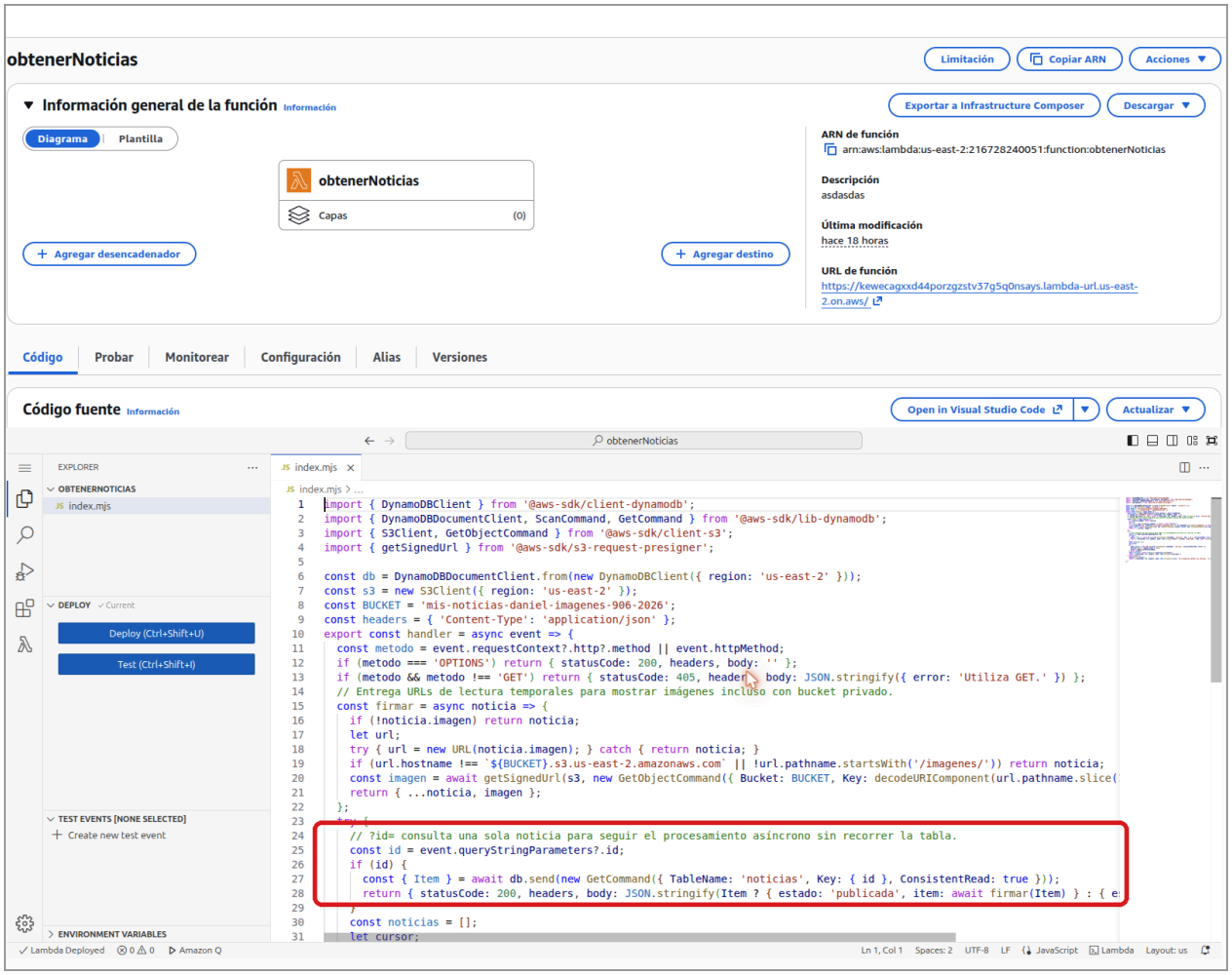


Figura 20. Código desplegado. Las líneas 24 a 28 atienden la consulta por id.

Comprobación. Abre en el navegador la URL de la función con un id que no existe:

```
https://TU-URL-DE-OBTENER.lambda-url.us-east-2.on.aws/?id=00000000-0000-4000-8000-000000000000
```

Debe responder `{"estado":"pendiente","id":"00000000-0000-4000-8000-000000000000"}` (figura 21). Sin `?id=`, debe seguir devolviendo la lista de noticias como antes.

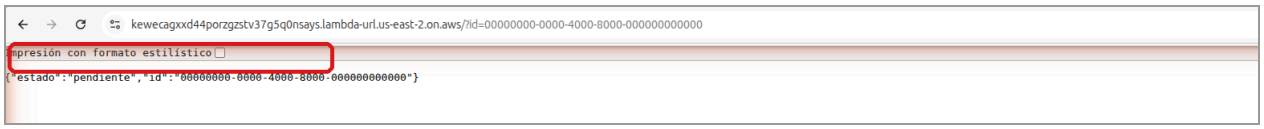


Figura 21. Respuesta de `obtenerNoticias?id=…` en el navegador para un id que no existe (comprobada el 4 de noviembre).

8. Publicar el nuevo sitio

El `main.js` nuevo cambia el envío de noticias con imagen: manda los datos a la función de prefirmadas, sube la imagen con la URL que recibe y consulta el estado cada 2 segundos, hasta 10 veces. **Ya no llama a `agregarNoticia` cuando hay imagen**; las noticias sin imagen la siguen usando, porque no generan evento de S3.

1. En `js/main.js`, pon las URL de tus funciones (líneas 760 a 762).
2. En **S3 › bucket del sitio › js/ › Cargar**, sube `main.js`.
3. En `index.html`, cambia el texto después de `?v=` en la etiqueta del script y sube también `index.html`. **Sin este cambio, el navegador sigue usando el `main.js` anterior que tiene en caché**; a nosotros nos pasó.

```
<script src="js/main.js?v=noticias-s3-evento-1" defer></script>
```

Estas son las partes de `main.js` que intervienen. El archivo completo, con el resto de la página, está en `codigo/main.js`.

main.js (líneas 758–763) — URL de las funciones

```
/* Integración AWS. Consulta INTEGRACION-S3.md para el contrato de las Lambda. */
const API_NOTICIAS = Object.freeze({
  obtener: 'https://kewecagxxd44porzgzstv37g5q0nsays.lambda-url.us-east-2.on.aws/',
  agregar: 'https://ufepu6szpto7mzedlr4isb3uiu0rzubq.lambda-url.us-east-2.on.aws/',
  prefirmar: 'https://krveeq3gnhqdy3r3kuyfhubrim0olhaw.lambda-url.us-east-2.on.aws/'
});
```

main.js (líneas 858–978) — subida, consulta del estado y envío del formulario

```
// Flujo asíncrono: la Lambda de prefirmadas guarda pendientes/ID.json y la procesadora S3 escribe en DynamoDB.
async function subirImagenConNoticia(archivo, noticia) {
  const datos = await solicitarJSON(API_NOTICIAS.prefirmar, {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ ...noticia, tipo: archivo.type })
  });
  const uploadUrl = urlSegura(datos.uploadUrl);
  if (!uploadUrl || datos.noticiaId !== noticia.id) throw new Error('La Lambda de prefirmadas debe devolver uploadUrl y el mismo noticiaId.');
  let respuesta;
  try {
    respuesta = await fetch(uploadUrl, {
      method: 'PUT', headers: { 'Content-Type': archivo.type }, body: archivo,
```

```
870       signal: AbortSignal.timeout(120000)
871     });
872   } catch (error) {
873     console.error('Falló la subida de la imagen a S3', error);
874     throw new Error('No se pudo completar la subida de la imagen a S3. Revisa CORS del bucket de imágenes y la
    vigencia de la URL prefirmada.');
875   }
876   if (!respuesta.ok) throw new Error(`No se pudo subir la imagen a S3 (HTTP ${respuesta.status}).`);
877 }
878
879 // Consulta por id durante un tiempo limitado; una subida exitosa no implica que la noticia ya esté guardada.
880 async function esperarProcesamiento(id, intentos = 10, esperaMs = 2000) {
881   const url = new URL(API_NOTICIAS.obtener);
882   url.searchParams.set('id', id);
883   for (let i = 0; i < intentos; i++) {
884     await new Promise(r => setTimeout(r, esperaMs));
885     try {
886       const datos = await solicitarJSON(url.href, { cache: 'no-store' });
887       if (datos.estado === 'publicada') return true;
888     } catch (error) { console.warn('Consulta de estado fallida', error); }
889   }
890   return false;
891 }
892
893 // Identificador de noticia compatible con sitios S3 servidos por HTTP.
894 function crearIdNoticia() {
895   if (typeof globalThis.crypto?.randomUUID === 'function') return globalThis.crypto.randomUUID();
896   const bytes = new Uint8Array(16);
897   if (typeof globalThis.crypto?.getRandomValues === 'function') {
898     globalThis.crypto.getRandomValues(bytes);
899   } else {
900     // Se utiliza para identificar noticias, no para autenticación ni secretos.
901     for (let i = 0; i < bytes.length; i++) bytes[i] = Math.floor(Math.random() * 256);
902   }
903   bytes[6] = (bytes[6] & 0x0f) | 0x40;
904   bytes[8] = (bytes[8] & 0x3f) | 0x80;
905   const hex = Array.from(bytes, b => b.toString(16).padStart(2, '0')).join('');
906   return `${hex.slice(0, 8)}-${hex.slice(8, 12)}-${hex.slice(12, 16)}-${hex.slice(16, 20)}-${hex.slice(20)}`;
907 }
908
909 function prepararPublicacion() {
910   const formulario = document.getElementById('formulario-noticia');
911   if (!formulario) return;
912   const campos = formulario.elements;
913   const hoy = new Date();
914   const fechaHoy = `${hoy.getFullYear()}-${String(hoy.getMonth() + 1).padStart(2, '0')}-${String(hoy.getDate
    ()).padStart(2, '0')}`;
915   campos.fecha.value = fechaHoy;
916   let enviando = false;
917   let imagenSubida = false;
918   let idPendiente = null;
919   campos.archivo.addEventListener('change', () => { imagenSubida = false; idPendiente = null; });
920   formulario.addEventListener('submit', async evento => {
921     evento.preventDefault();
922     if (enviando || !formulario.reportValidity()) return;
923     const estado = document.getElementById('publicacion-estado');
924     const archivo = campos.archivo.files[0];
925     if (['titulo', 'categoria', 'detalle'].some(nombre => !campos[nombre].value.trim())) {
926       estado.textContent = 'Completa título, categoría y detalle.'; return;
927     }
928     if (archivo && (!['image/jpeg', 'image/png', 'image/webp'].includes(archivo.type) || archivo.size === 0 ||
    archivo.size > 10 * 1024 * 1024)) {
929       estado.textContent = 'Selecciona una imagen JPG, PNG o WebP de máximo 10 MB.'; return;
930     }
931     if (archivo && !campos.alt.value.trim()) {
```

```
932       estado.textContent = 'Agrega una descripción de la imagen.'; campos.alt.focus(); return;
933     }
934     enviando = true;
935     document.getElementById('campos-noticia').disabled = true;
936     let guardada = false;
937     try {
938       if (!idPendiente) idPendiente = crearIdNoticia();
939       const noticia = {
940         id: idPendiente, titulo: campos.titulo.value.trim(), detalle: campos.detalle.value.trim(),
941         fecha: campos.fecha.value, categoria: campos.categoria.value.trim(), alt: campos.alt.value.trim()
942       };
943       const mensaje = 'Noticia publicada correctamente.';
944       if (archivo) {
945         if (!imagenSubida) {
946           estado.textContent = 'Subiendo imagen…';
947           await subirImagenConNoticia(archivo, noticia);
948           imagenSubida = true;
949         }
950         estado.textContent = 'Imagen recibida. Procesando la noticia…';
951         if (!await esperarProcesamiento(idPendiente)) {
952           // No se reenvía: un reintento crearía otra subida. La procesadora puede terminar más tarde.
953           estado.textContent = 'La imagen se subió y la noticia sigue en procesamiento. Pulsa Actualizar noticias en unos momentos.';
954           formulario.reset(); campos.fecha.value = fechaHoy; imagenSubida = false; idPendiente = null;
955           return;
956         }
957       } else {
958         estado.textContent = 'Guardando noticia…';
959         await solicitarJSON(API_NOTICIAS.agregar, {
960           method: 'POST', headers: { 'Content-Type': 'application/json' },
961           body: JSON.stringify({ ...noticia, imagen: '' })
962         });
963       }
964       guardada = true;
965       formulario.reset();
966       campos.fecha.value = fechaHoy;
967       imagenSubida = false;
968       idPendiente = null;
969       const actualizada = await cargarNoticias();
970       estado.textContent = actualizada ? mensaje : 'Noticia guardada. No se pudo actualizar la lista; pulsa Actualizar noticias.';
971     } catch (error) {
972       estado.textContent = guardada ? 'Noticia guardada. Actualiza la lista.' : `No se confirmó la publicación. ${error.message} Tus datos se conservaron.`;
973     } finally {
974       enviando = false;
975       document.getElementById('campos-noticia').disabled = false;
976     }
977   });
978 }
```

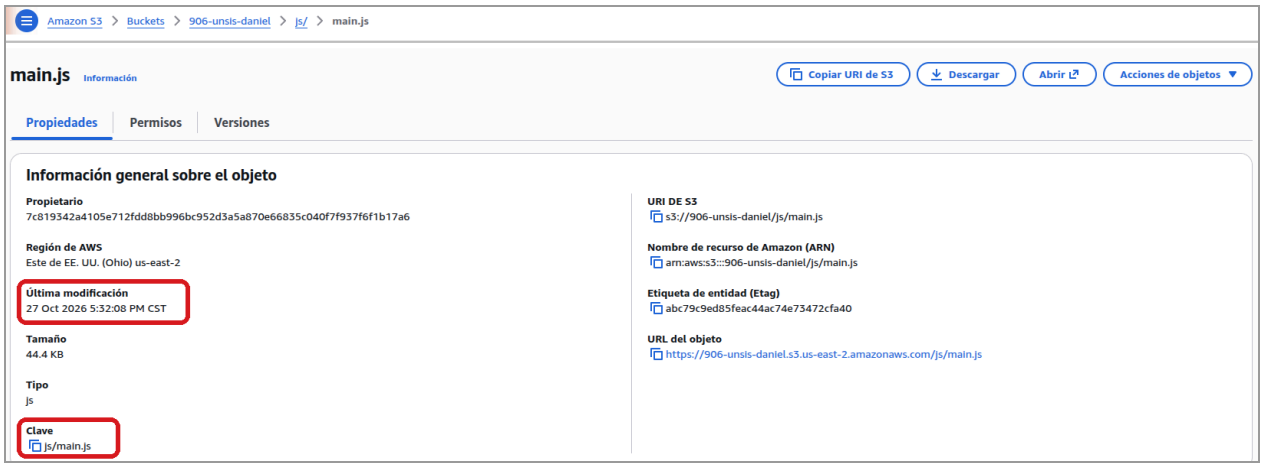


Figura 22. S3 › bucket del sitio › js/main.js. Fecha de la última modificación y clave del objeto.

Comprobación. Abre el sitio, pulsa `Ctrl`+`F5` y, en las herramientas del navegador (`F12` › Red), confirma que se carga `main.js?v=` con el valor nuevo.

9. Configurar reintentos y retención de logs

Reintentos. S3 invoca la función de forma asíncrona: si la función falla, Lambda la vuelve a intentar. En **Configuración › Invocación asíncrona › Editar**, escribe **1** hora, 0 min y 0 s en «Antigüedad máxima del evento», **2** en «Reintentos», deja la cola de mensajes fallidos en «Ninguno» y pulsa **Guardar**.



Figura 23. Configuración › Invocación asíncrona › Editar. Antigüedad máxima de 1 hora, 2 reintentos y sin cola de mensajes fallidos.

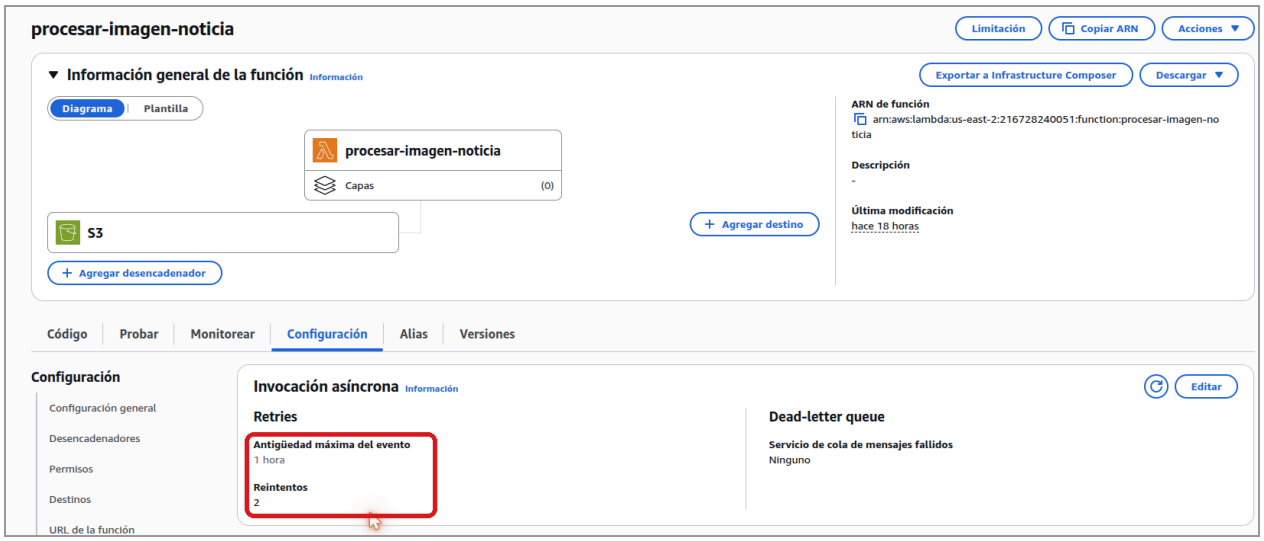


Figura 24. Configuración › Invocación asíncrona después de guardar. No configuramos cola de mensajes fallidos (DLQ).

Retención de logs. Por defecto, CloudWatch guarda los logs para siempre, y el almacenamiento se cobra. En **CloudWatch › Registros › Administración de registros**, abre `/aws/lambda/procesar-imagen-noticia` y, en **Acciones › Editar configuración de retención**, elige **Días 3** en «Marcar eventos como vencidos después de» y pulsa **Guardar**. El grupo existe desde la primera ejecución de la función; si no aparece, ejecuta antes la comprobación de la función procesadora.

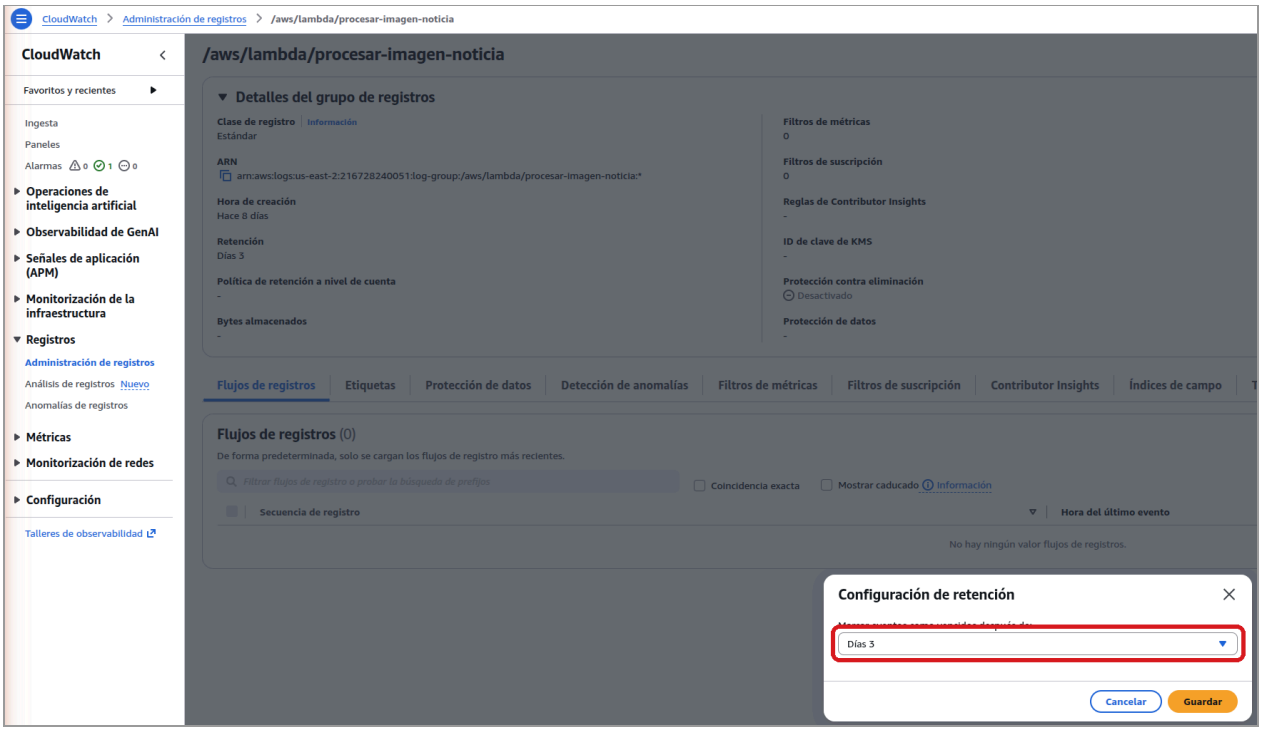


Figura 25. Acciones › Editar configuración de retención. Días 3.

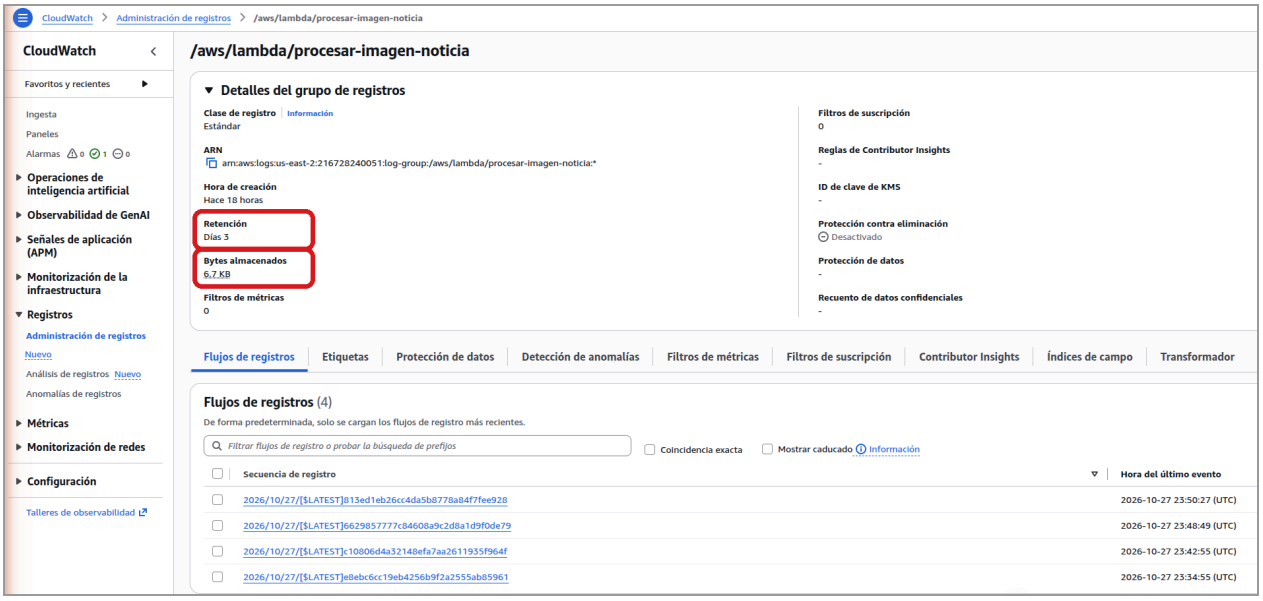


Figura 26. CloudWatch › grupo de logs de la procesadora. Retención de 3 días y 6.7 KB almacenados tras todas nuestras pruebas.

La función escribe una línea JSON por etapa (`inicio`, `validado`, `guardado`, `duplicado`, `validacion_fallida`, `omitido_sin_formato` y `fallo`) con `requestId`, `bucket`, `key`, `noticiaId` y `duracionMs`. No registra el texto de la noticia ni ninguna URL firmada.

Comprobación. En Invocación asíncrona se lee «Reintentos 2» y en el grupo de logs, «Retención: Días 3».

10. Crear la alarma

La señal que vigilamos es la métrica `Errors` que Lambda publica sola en CloudWatch, sin costo: cuenta las ejecuciones que terminan con error. El asistente tiene cuatro pasos.

10.1. Paso 1: métrica y condiciones

1. En **CloudWatch › Alarmas**, pulsa **Crear alarma**. Deja «Métricas» y «Clásico» y pulsa **Seleccione una métrica** (figura 27).
2. En el selector, entra a **Lambda › Por nombre de función**, busca `procesar-imagen-noticia`, marca la fila `Errors` y pulsa **Seleccionar una métrica** (figura 28).
3. En **Métrica**, elige la estadística **Suma** y el periodo **1 minuto**.
4. En **Condiciones**: tipo **Estático**, **Mayor/Igual** que **1**.
5. Abre **Configuración adicional**: **1 de 1** puntos de datos y **Tratar los datos que faltan como correctos**; si no, la alarma queda en «Datos insuficientes» cuando no hay publicaciones. Pulsa **Siguiente** (figura 29).

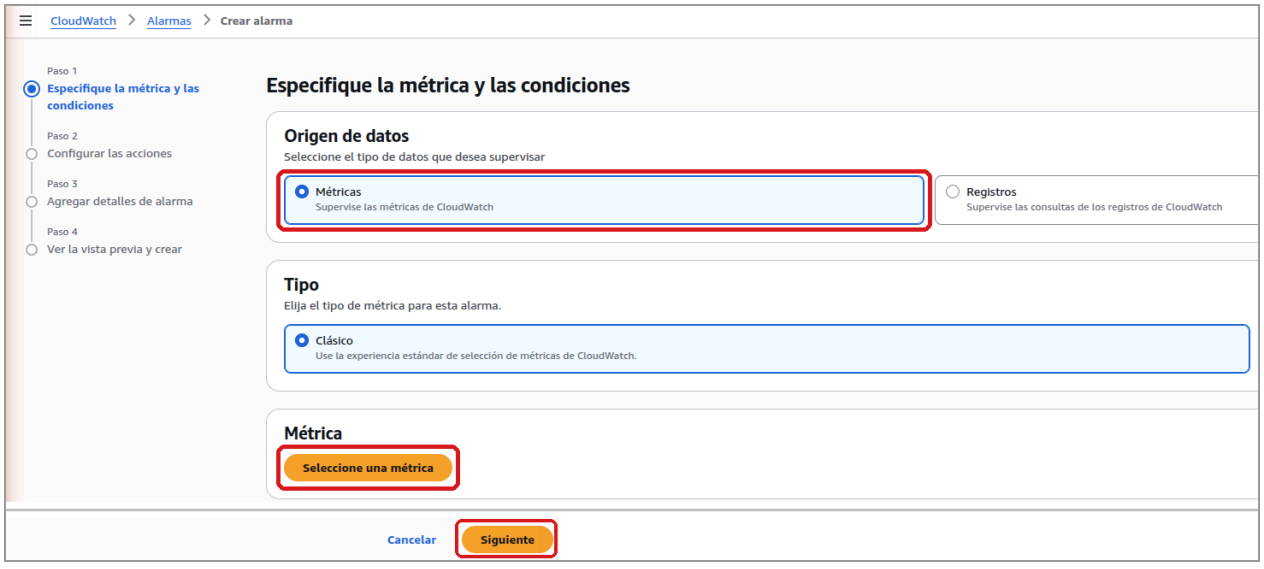


Figura 27. CloudWatch › Alarmas › Crear alarma. Origen «Métricas», tipo «Clásico» y el botón para elegir la métrica. Abajo, el botón «Siguiente» del extremo derecho de la página.

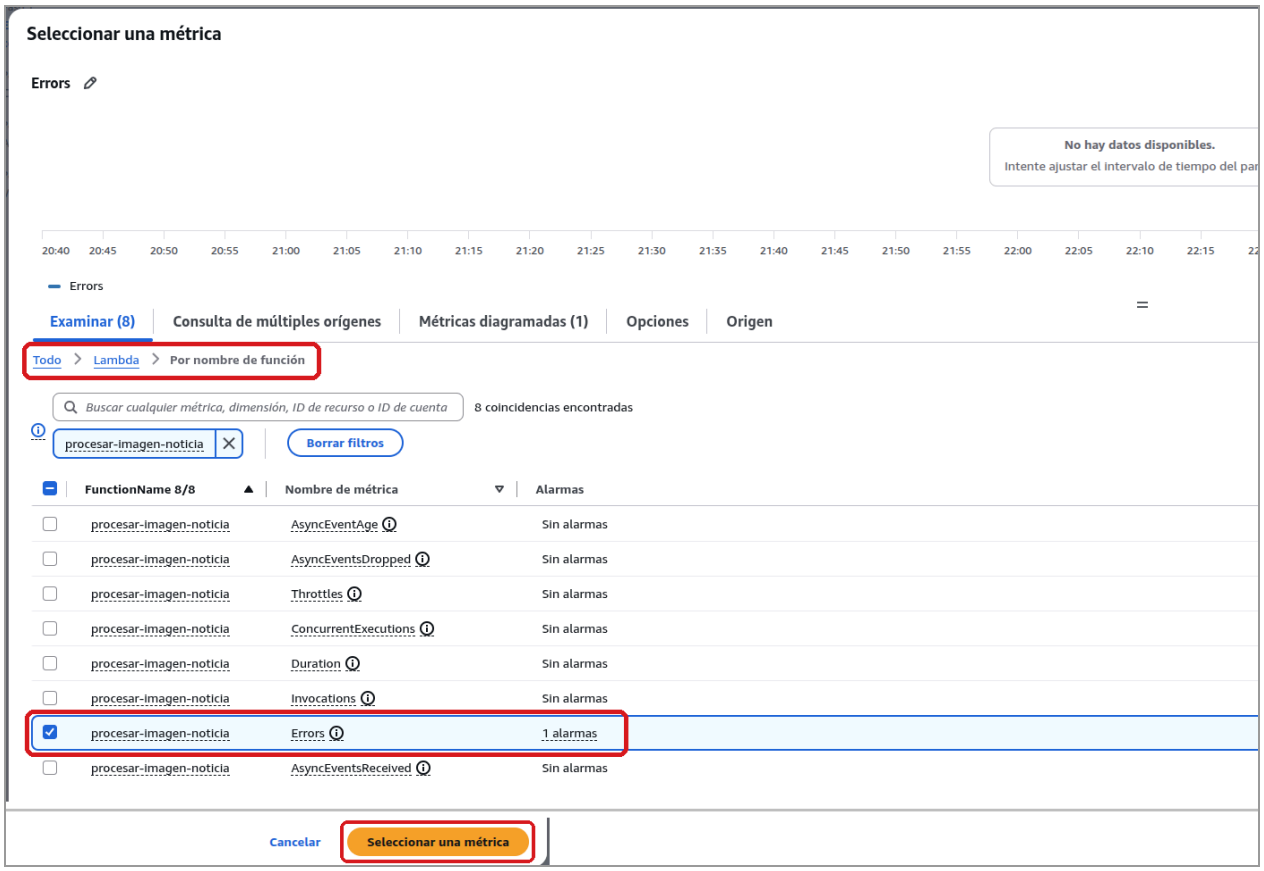


Figura 28. Seleccionar una métrica. Lambda › Por nombre de función, filtrado por la procesadora y con `Errors` marcada. En nuestra cuenta la columna dice «1 alarmas» porque la alarma ya existe; en una cuenta nueva dirá «Sin alarmas».

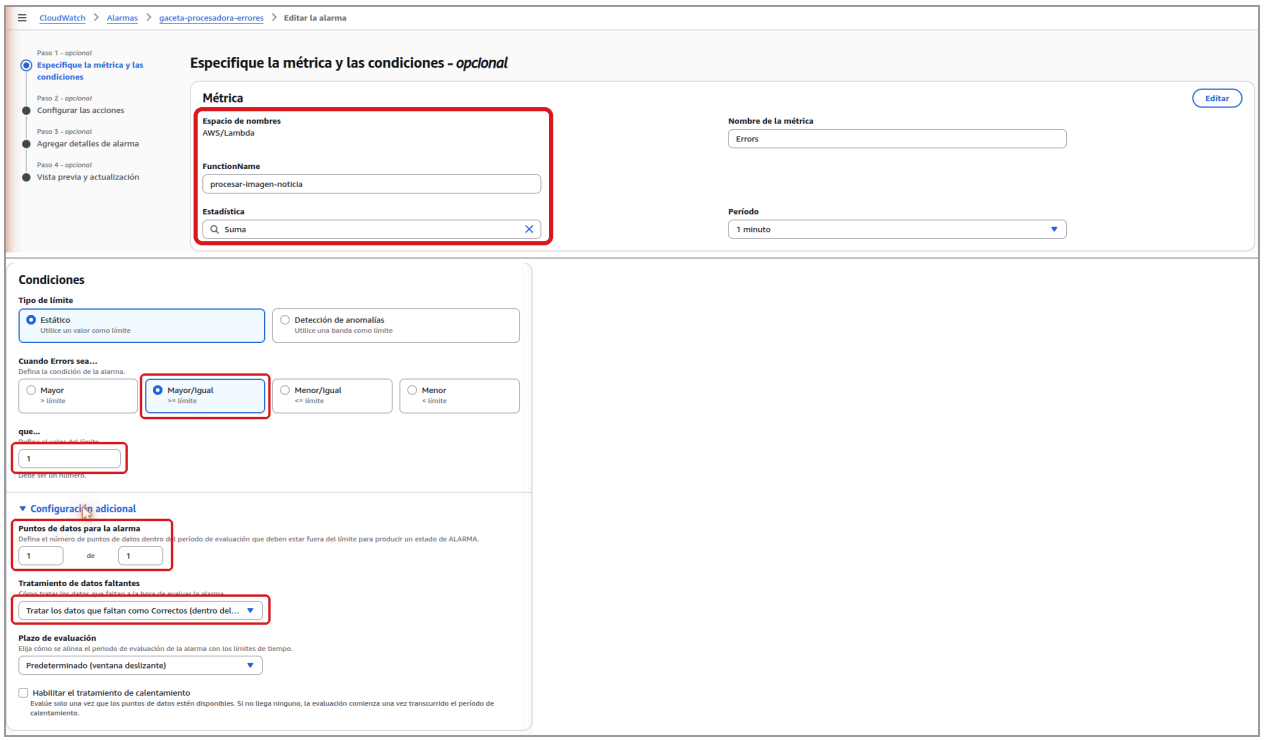


Figura 29. Arriba, la métrica: `AWS/Lambda`, `Errors`, la función y la suma por minuto. Abajo, las condiciones y la configuración adicional.

10.2. Paso 2: acciones

No agregues ninguna acción (sin SNS no hay correos ni suscripciones, y no hay costo extra). Pulsa **Siguiente**; la consola avisa que no hay acciones configuradas: pulsa **Proceder sin acciones**.

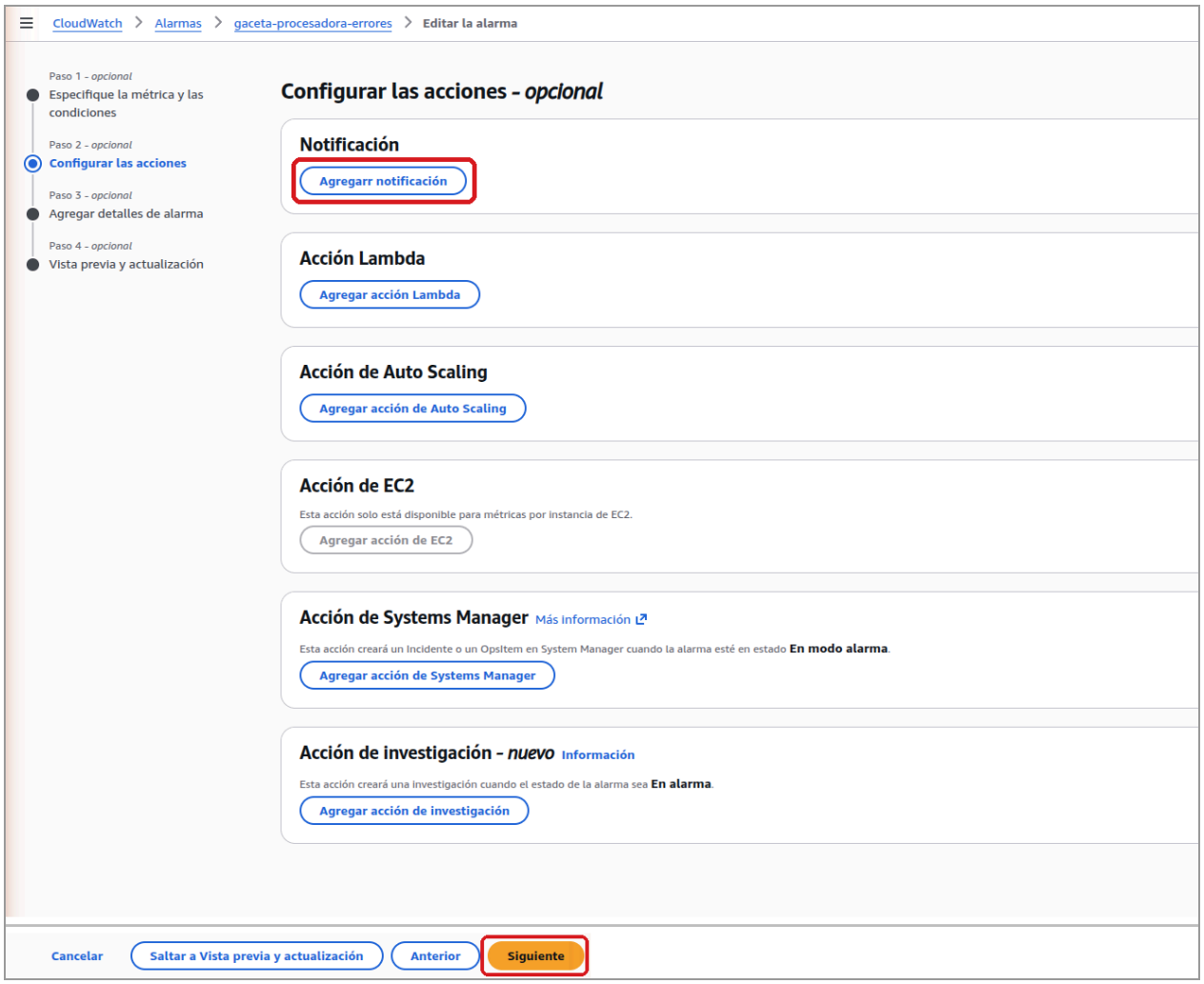


Figura 30. Paso 2 · Configurar las acciones. Se deja todo vacío y se pulsa «Siguiente».

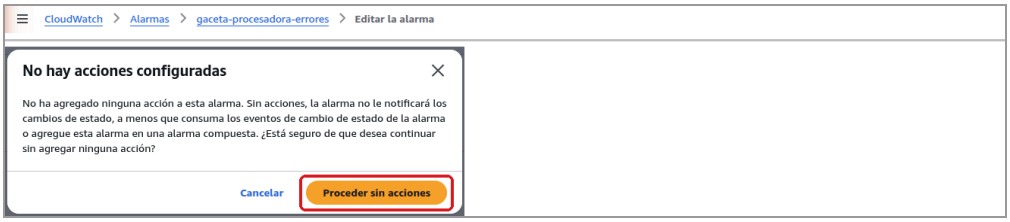


Figura 31. Aviso «No hay acciones configuradas»: se continúa con «Proceder sin acciones».

10.3. Paso 3: nombre y descripción

Escribe el nombre `gaceta-procesadora-errores`, una descripción y las etiquetas `proyecto`, `equipo` y `entorno`. Pulsa **Siguiente**.

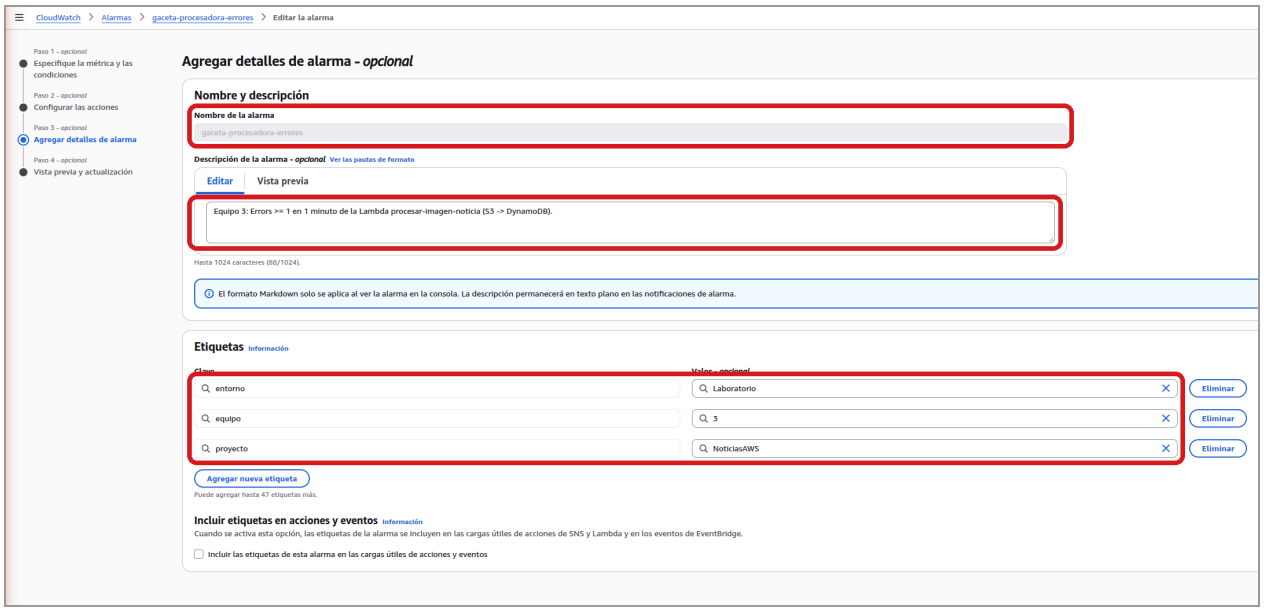


Figura 32. Paso 3 · Agregar detalles de alarma. Nombre, descripción y etiquetas. Esta captura y las dos anteriores se tomaron con **Acciones › Editar** sobre nuestra alarma (y se canceló sin guardar), por eso el nombre aparece gris: al crearla, el campo está vacío y se puede escribir.

10.4. Paso 4: vista previa

Revisa el resumen y pulsa **Crear alarma** (en la edición, el botón dice «Actualizar alarma»).

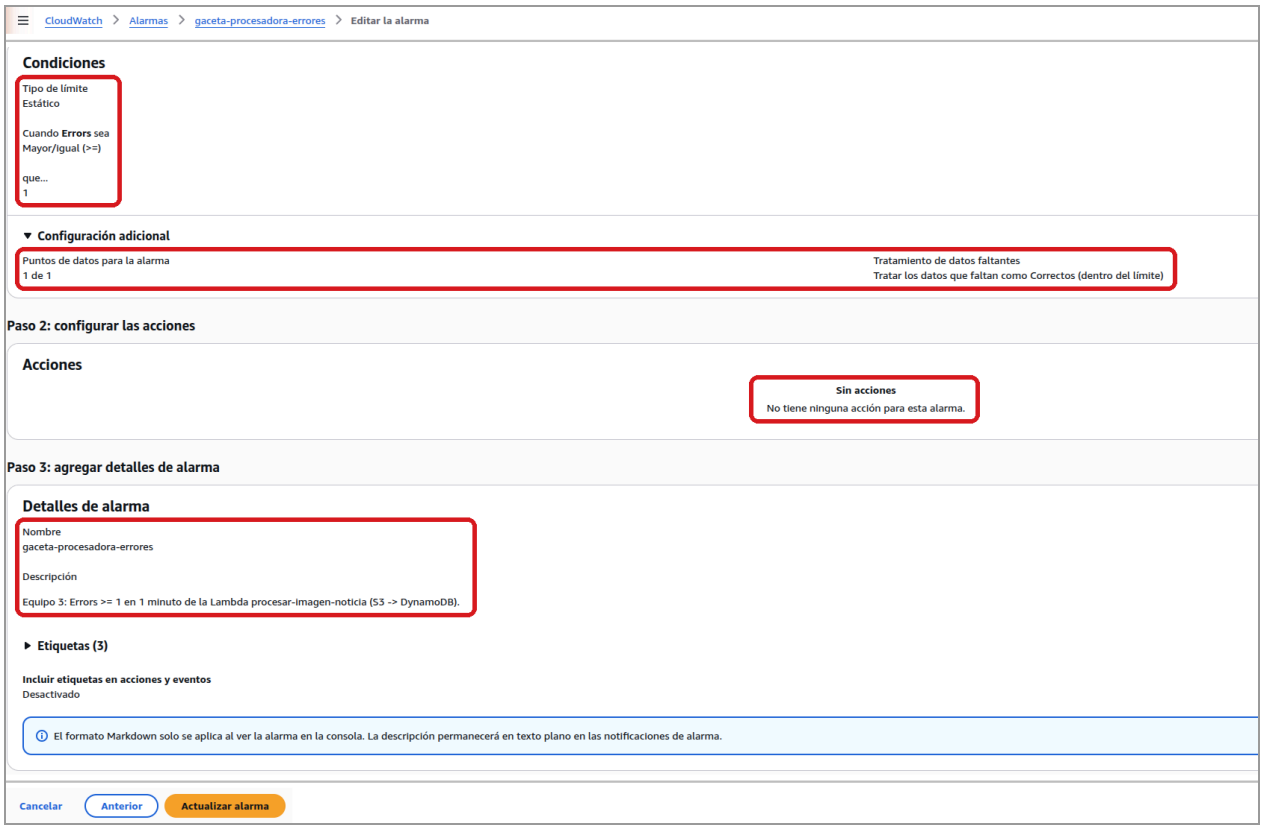


Figura 33. Paso 4 · Vista previa. Condición estática ≥ 1, 1 de 1 puntos de datos, datos faltantes como correctos, sin acciones, y el nombre y la descripción.

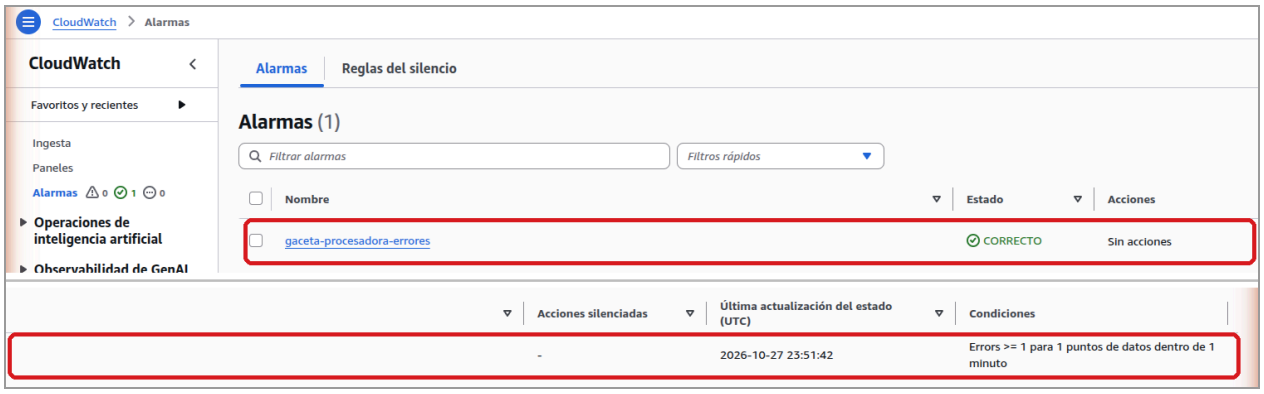


Figura 34. CloudWatch › Alarmas. La alarma en estado «Correcto», sin acciones, con la condición «Errors >= 1 para 1 puntos de datos dentro de 1 minuto». La lista se partió en dos franjas para que se lea.

Comprobación. La alarma aparece en estado **CORRECTO**. Al principio puede pasar unos minutos en «Datos insuficientes».

11. Etiquetar los recursos

Para identificar el costo de lo que creamos, etiquetamos la procesadora (**Configuración** › **Etiquetas**), su rol (al crearlo) y la alarma (pestaña **Etiquetas**) con `proyecto=NoticiasAWS`, `equipo=3` y `entorno=Laboratorio`. Los recursos compartidos no se etiquetaron.

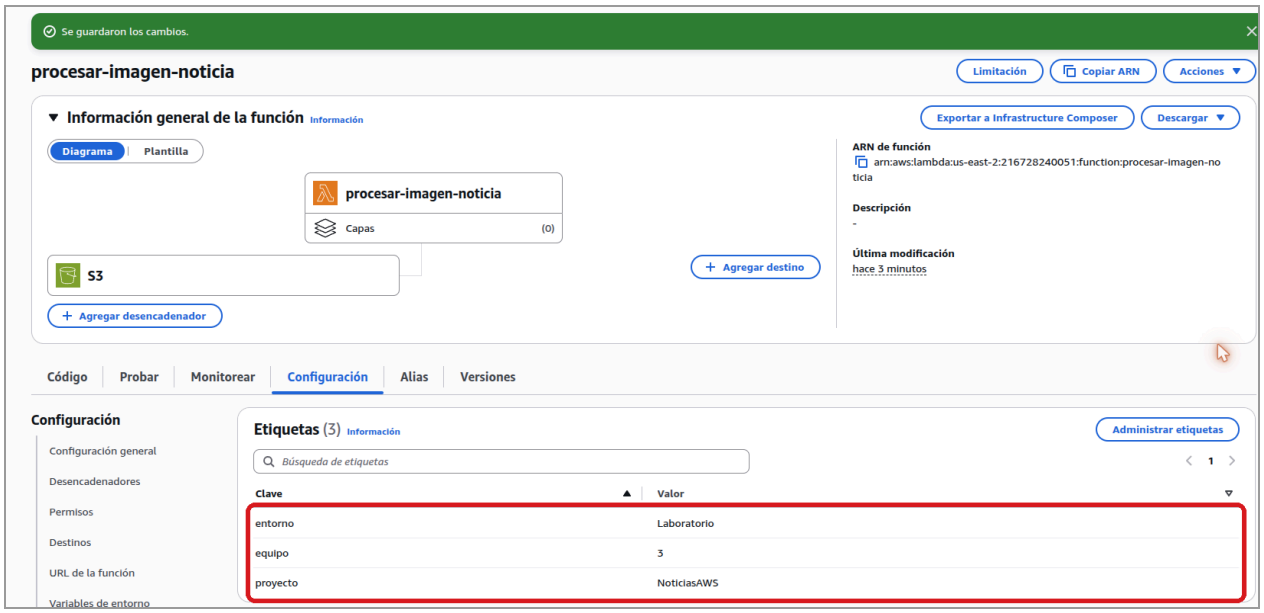


Figura 35. Lambda › procesar-imagen-noticia › Configuración › Etiquetas. Etiquetas de la procesadora.

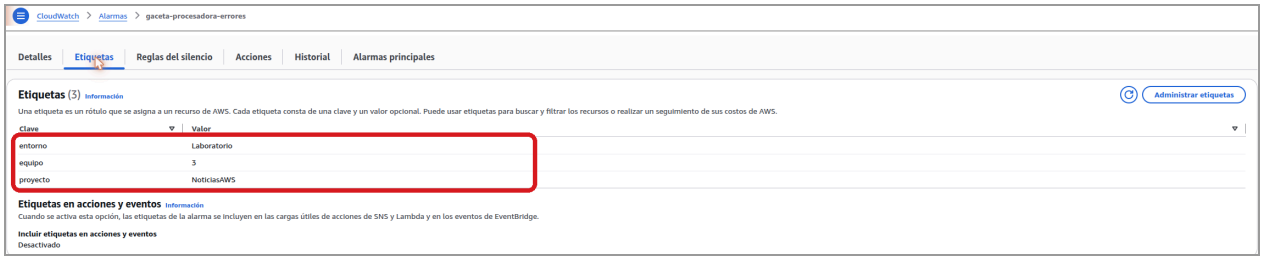


Figura 36. CloudWatch › Alarmas › gaceta-procesadora-errores › Etiquetas. Las mismas tres etiquetas.

12. Verificación de punta a punta

Con todo configurado, se comprueba el flujo en condiciones normales y provocando fallos a propósito. Las pruebas P01, P02, P04 y P08 se hacen desde el sitio; P03 y P07 con la pestaña **Probar** de la procesadora y los eventos de la carpeta `pruebas/` del repositorio, cambiando el bucket y el id por los tuyos. La tabla 4 resume lo que debe pasar; nosotros obtuvimos ese resultado en todas.

Tabla 4. Pruebas y resultado esperado.

Prueba	Qué se hace	Resultado esperado
P01	Publicar una noticia con imagen desde el sitio	«Noticia publicada correctamente»; en el log, `inicio → validado → guardado`; en Red, ninguna llamada a `agregarNoticia`
P02	Revisar la tabla y recargar el sitio	La noticia está en DynamoDB con `origen = s3-evento` y se ve con su imagen
P03	Evento de una imagen sin pendiente	Ejecución sin error; log `validacion_fallida` por `NoSuchKey`; no se crea noticia
P04	Poner `TABLA` a una tabla inexistente y publicar	3 intentos fallidos con el mismo `requestId`; al restaurar `TABLA` y reprocesar, se guarda con el mismo id
P05	Ver **Monitorear** después de P04	La métrica `Errors` sube; en eventos asíncronos, 3 recibidos y 1 eliminado
P06	Ver la alarma durante y después de P04	Pasa a «En modo alarma» y vuelve a «Correcto» unos 5 minutos después
P07	Reenviar el evento de la imagen de P01	Log `duplicado`; la tabla conserva un solo registro
P08	Publicar mientras la tabla está «caída»	El sitio avisa que sigue en procesamiento, sin reenviar; tras la recuperación aparece una sola vez

12.1. P01 y P02: publicación normal

Publica una noticia con imagen desde «Crear noticia». El sitio dice «Imagen recibida. Procesando la noticia…» y, uno o dos segundos después, «Noticia publicada correctamente».

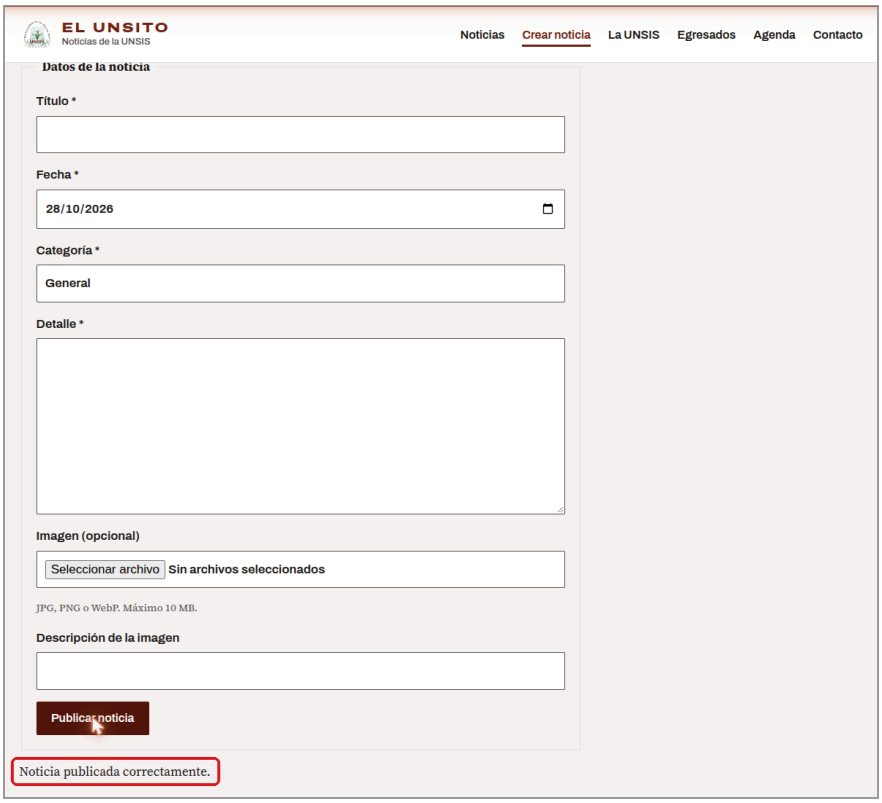


Figura 37. Formulario del sitio después de publicar.

En **CloudWatch › grupo de logs de la procesadora**, abre el flujo más reciente: se ven las tres etapas de la publicación y la duración.



Figura 38. Logs de P01: `inicio`, `validado` y `guardado`; el procesamiento completo duró 1 561 ms.

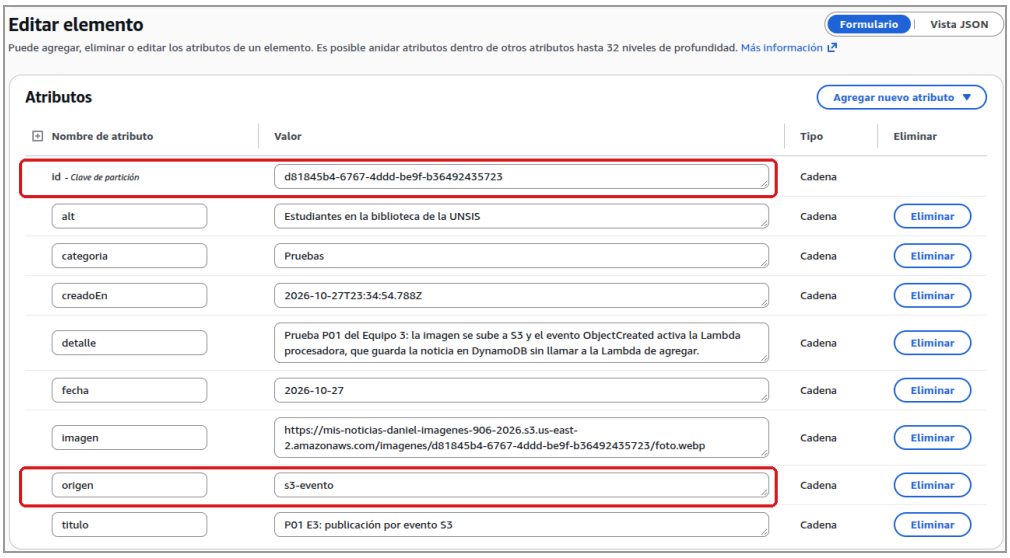


Figura 39. DynamoDB › Explorar elementos. La noticia tiene el mismo id y `origen = s3-evento`, prueba de que la guardó la procesadora.

12.2. P03: imagen sin datos

En **Probar**, ejecuta el evento `evento-s3-sin-pendiente.json` (el mismo de la comprobación de la función procesadora).

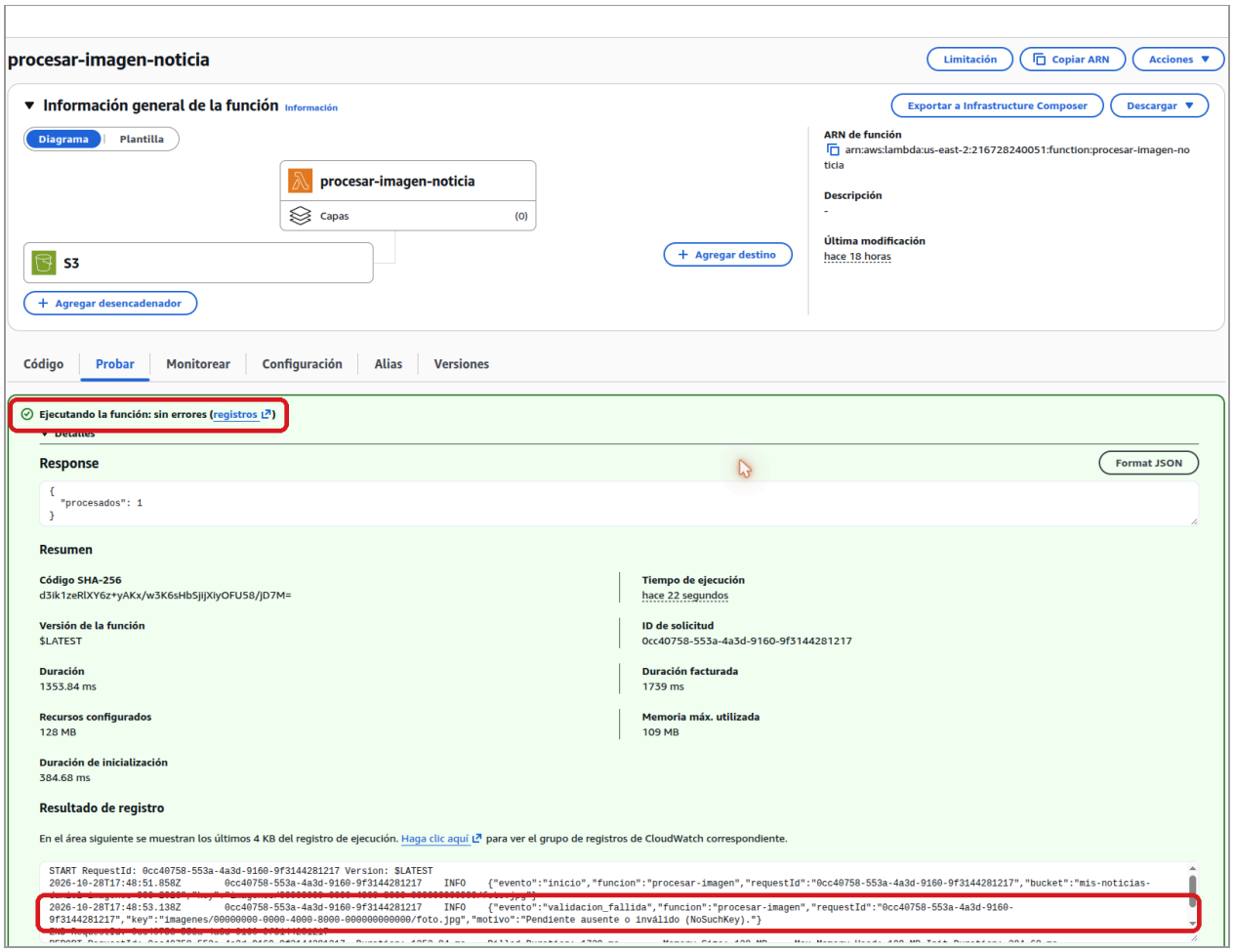


Figura 40. «Ejecutando la función: sin errores» y log `validacion_fallida` con el motivo «Pendiente ausente o inválido (NoSuchKey)».

12.3. P04 a P06: falla de la base de datos, métrica y alarma

Para simular que DynamoDB falla sin dañar la tabla real, cambia la variable `TABLA` por un nombre que no exista, por ejemplo `noticias-falla-lab`, y publica una noticia con imagen.

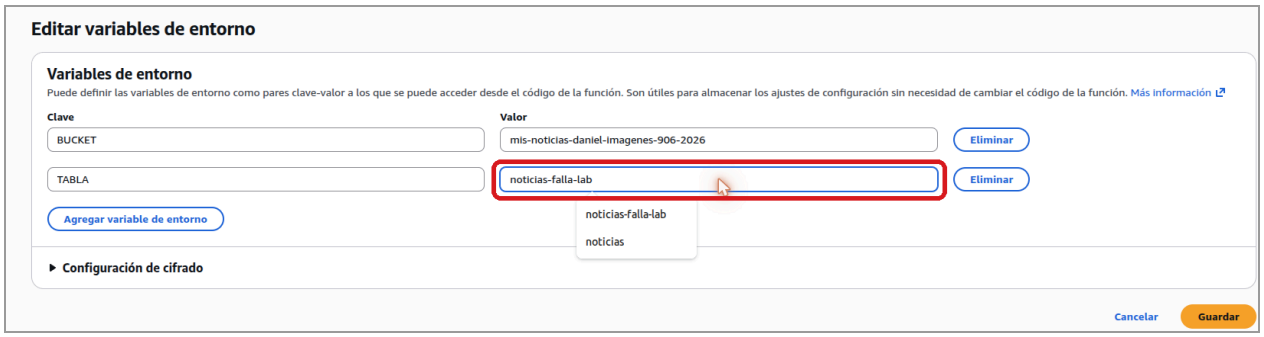


Figura 41. Variables de entorno › Editar. `TABLA = noticias-falla-lab`.

En el grupo de logs, filtra con `?fallo ?ResourceNotFoundException` más parte del id. Deben aparecer tres intentos con el mismo `requestId`: el original y los dos reintentos, separados por uno o dos minutos.

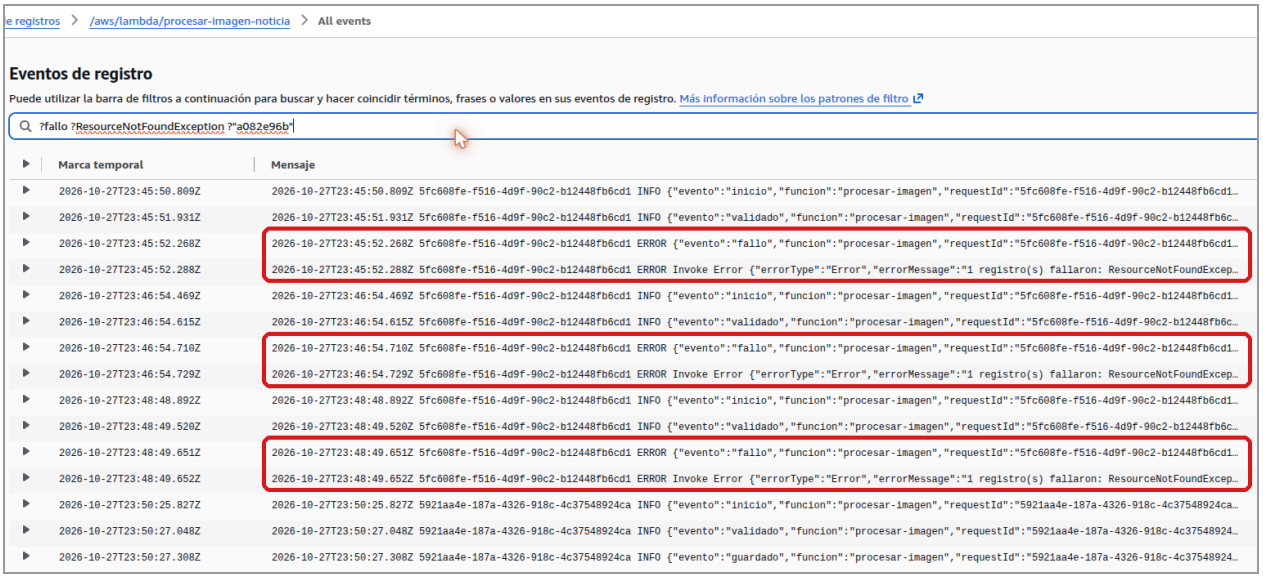


Figura 42. Tres errores con el mismo `requestId` (23:45:52, 23:46:54 y 23:48:49 UTC). Las últimas tres líneas son la recuperación.

Restaura `TABLA = noticias`. Si los reintentos ya se agotaron, reprocesa el evento en **Probar** con `evento-s3-recuperacion-p04.json` (con el id de tu noticia). La noticia se guarda con el mismo id.

pruebas/evento-s3-recuperacion-p04.json

```
{
  "Records": [{
    "eventSource": "aws:s3",
    "awsRegion": "us-east-2",
    "eventName": "ObjectCreated:Put",
    "s3": {
      "bucket": { "name": "mis-noticias-daniel-imagenes-906-2026" },
      "object": { "key": "imagenes/a082e96b-a1b7-40da-bade-06677ac730ca/foto.webp", "size": 48128 }
    }
  }]
}
```



Figura 43. Log `guardado` en la tabla `noticias` con el id de la noticia que había fallado.

En la pestaña **Monitorear** de la procesadora se ve la falla en las métricas nativas de Lambda.

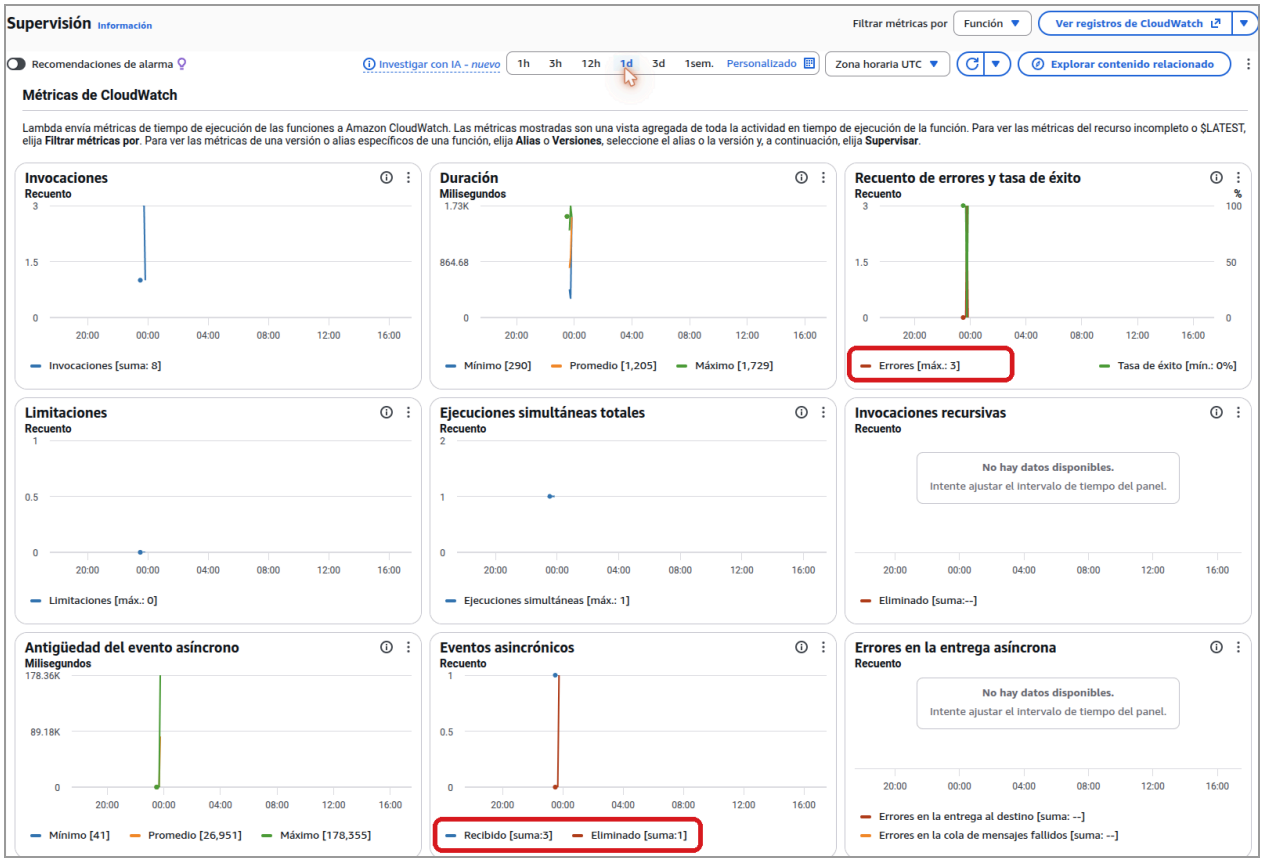


Figura 44. «Recuento de errores» con `Errors` y tasa de éxito mínima del 0 %, y «Eventos asincrónicos» con 3 recibidos y 1 eliminado tras agotar los reintentos.

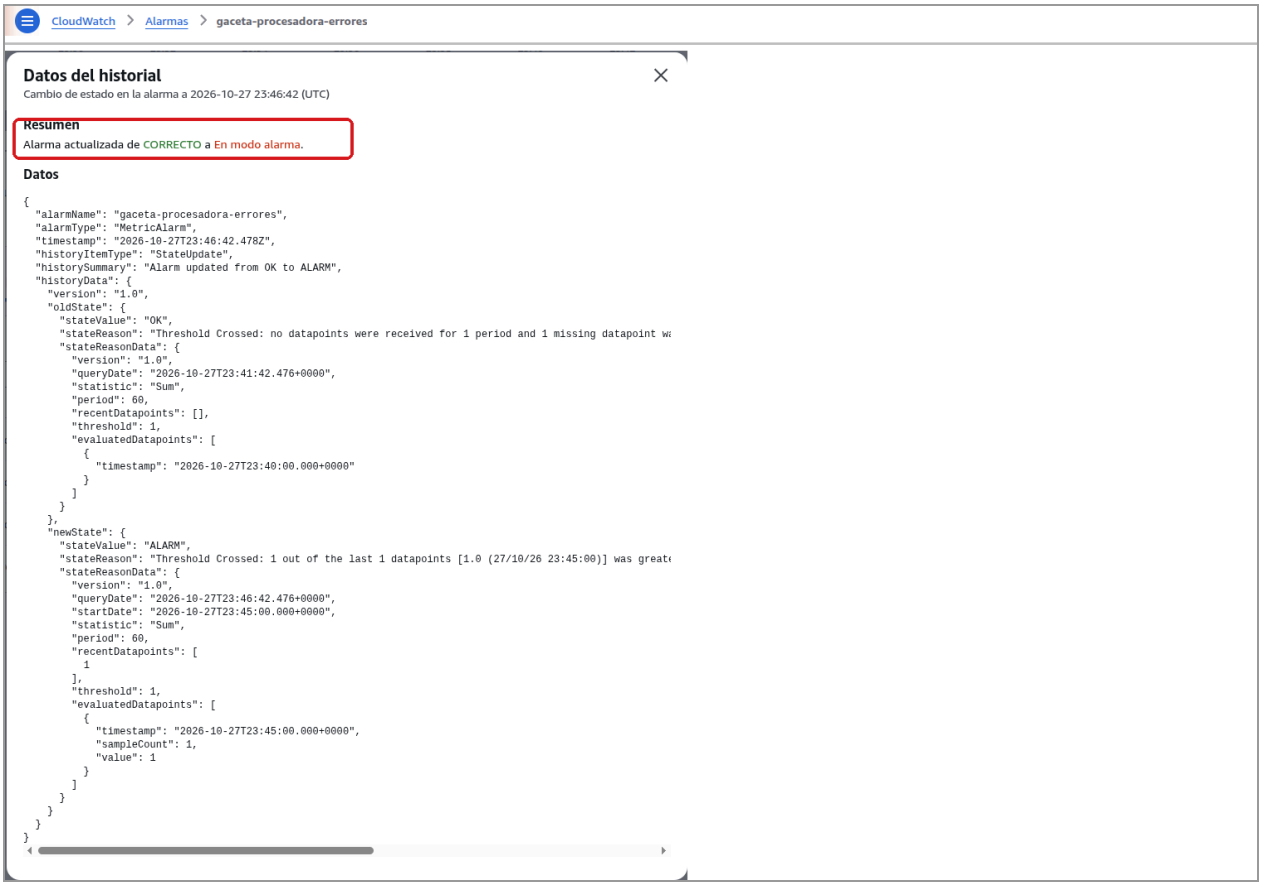


Figura 45. Alarma › Historial. Cambio de «CORRECTO» a «En modo alarma» a las 23:46:42 UTC.

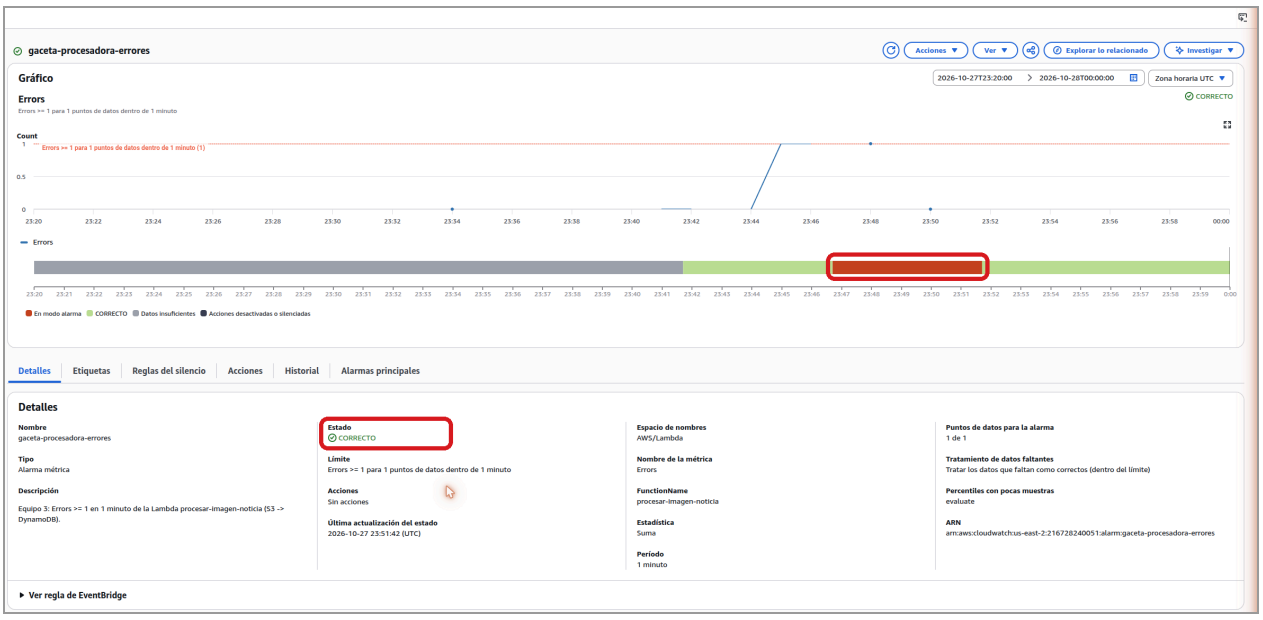


Figura 46. La franja roja marca el tiempo en alarma; a las 23:51:42 UTC vuelve a «CORRECTO».

12.4. P07: evento repetido

S3 puede entregar un mismo evento más de una vez. Ejecuta en **Probar** el evento de la imagen de P01 (`evento-s3-duplicado.json`, con tu id).

pruebas/evento-s3-duplicado.json

```
{
  "Records": [{
    "eventSource": "aws:s3",
    "awsRegion": "us-east-2",
    "eventName": "ObjectCreated:Put",
    "s3": {
      "bucket": { "name": "mis-noticias-daniel-imagenes-906-2026" },
      "object": { "key": "imagenes/d81845b4-6767-4ddd-be9f-b36492435723/foto.webp", "size": 86016 }
    }
  }]
}
```

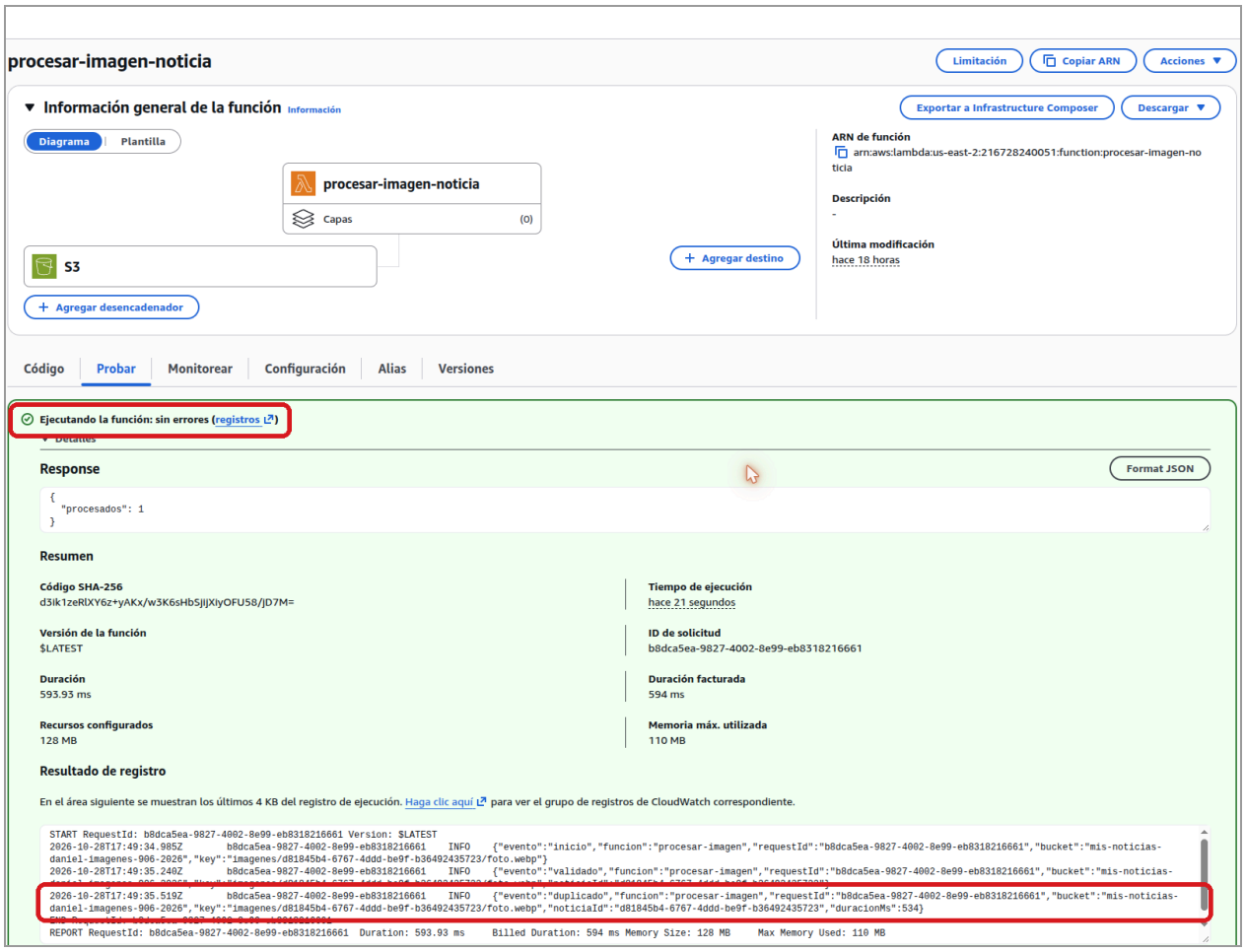


Figura 47. Ejecución sin errores y log `duplicado` para la misma noticia; la tabla conserva un solo registro.

12.5. P08: procesamiento lento

Repite la falla de P04 y publica desde el sitio. Tras 10 consultas (unos 20 segundos), el sitio avisa que la noticia sigue en procesamiento, sin volver a enviarla. Al restaurar `TABLA`, uno de los reintentos automáticos la guarda y aparece una sola vez.

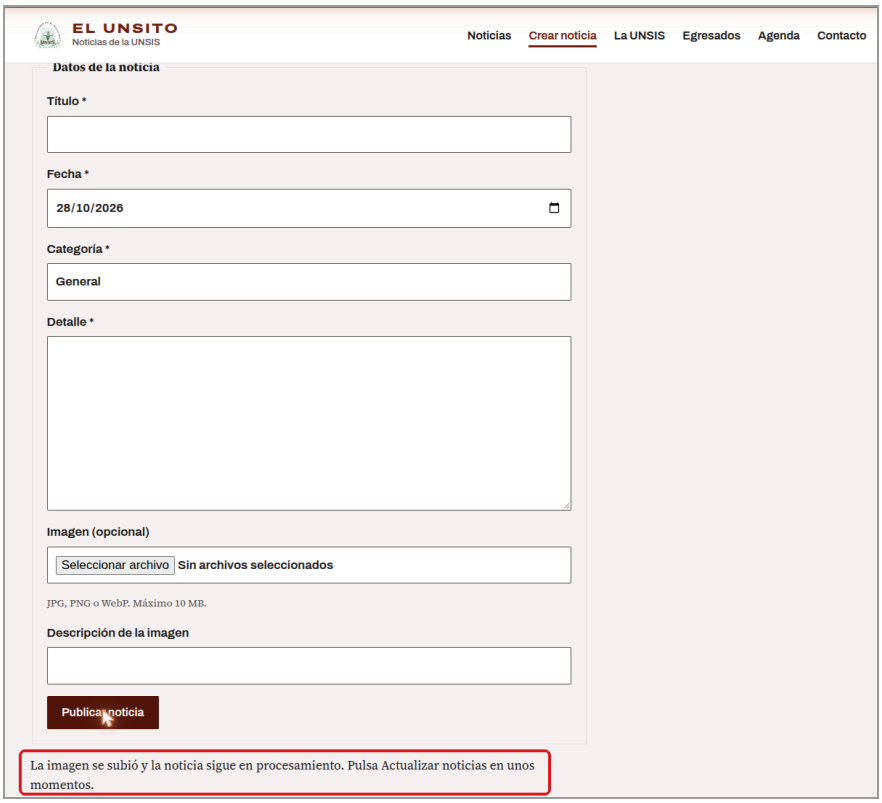


Figura 48. Mensaje del sitio mientras la noticia sigue pendiente.

12.6. Prueba del código sin AWS

Además de las pruebas en AWS, el repositorio incluye `documentacion/herramientas/prueba-local/`, que ejecuta las tres funciones de este manual en el navegador contra S3 y DynamoDB simulados. Comprueba 13 casos (P01–P04, P07, la consulta `?id=`, el modo `?file=` y la lista). Al revisar este manual, los 13 pasaron.

```
python3 -m http.server 8765      # desde la raíz del repositorio
# abrir http://127.0.0.1:8765/documentacion/herramientas/prueba-local/prueba.html
```

12.7. Revisión del estado en AWS

El 4 de noviembre de 2026, al revisar este manual, volvimos a abrir cada recurso en la consola (solo lectura: los formularios se abrieron y se cancelaron sin guardar). Todo seguía como lo describe el manual.

Tabla 5. Estado comprobado en AWS el 4 de noviembre de 2026.

Recurso	Qué se comprobó	Resultado
Lambda `procesar-imagen-noticia`	Tiempo de espera, variables, disparador y rol	**30 s; `BUCKET` y `TABLA = noticias`; S3 sobre el bucket de imágenes; `procesar-imagen-noticia-rol`**
Invocación asíncrona	Formulario de edición (figura 23)	**1 hora, 2 reintentos, sin DLQ**
Rol de IAM	Política insertada	**Solo `ProcesadoraMinimoPrivilegio`, idéntica al archivo del manual, 0 errores**
Grupo de logs	Retención y contenido (figura 49)	**Días 3; los flujos de las pruebas ya vencieron y se borraron solos**
Alarma	Estado y configuración (figuras 28 a 33)	**CORRECTO, condición ≥ 1 en 1 minuto, sin acciones**
`obtenerNoticias`	`?id=` de un id inexistente (figura 21)	**`{"estado":"pendiente",…}`**
Sitio publicado	Script que carga la página	**`main.js?v=noticias-s3-evento-1`, con `subirImagenConNoticia` y `esperarProcesamiento`**
Facturación	Créditos y costo por servicio (figuras 52 y 53)	**120.00 USD de créditos; CloudWatch 0.00 USD**

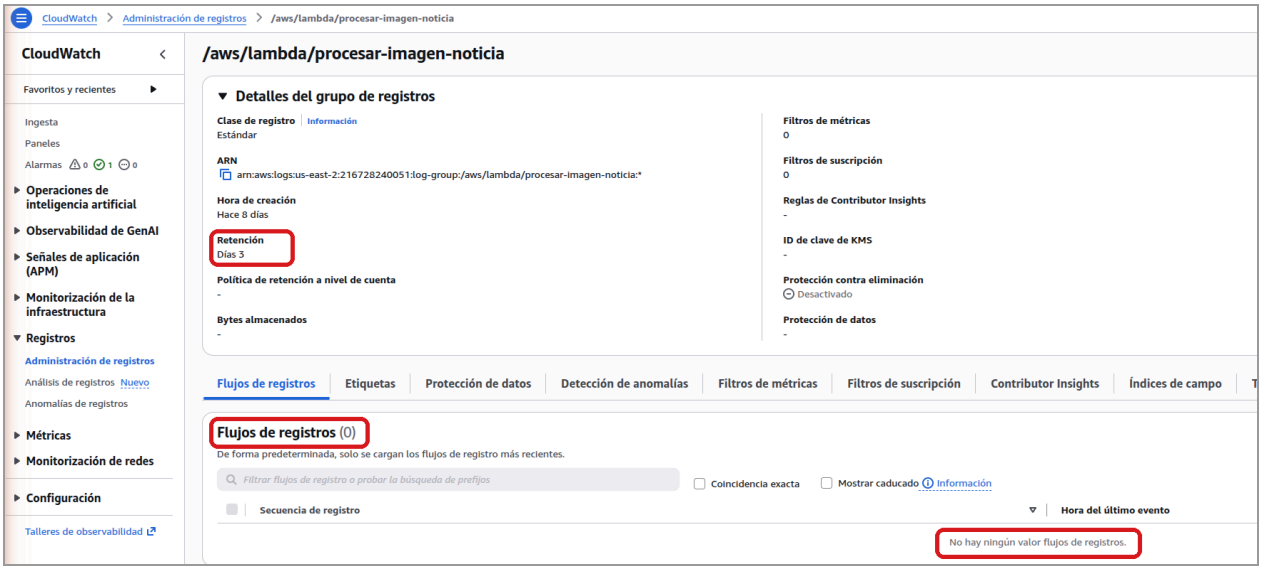


Figura 49. Grupo de logs el 4 de noviembre: retención de 3 días, «Flujos de registros (0)» y sin bytes almacenados. Los logs de las pruebas del 27 y 28 de octubre ya vencieron, así que no generan costo de almacenamiento.

Las pruebas P01 a P08 no se repitieron ese día, para no escribir datos nuevos en la tabla compartida; sus capturas son de la ejecución del 27 y 28 de octubre.

13. Costos

13.1. Cómo cobra CloudWatch

CloudWatch no tiene cuota fija: se paga por uso, por lo que se envía y se guarda. La tabla 6 resume los conceptos que tocan a este proyecto, con los precios de lista de la región us-east-2 publicados por AWS (consultados en noviembre de 2026; pueden cambiar). La capa gratuita se aplica cada mes y por cuenta.

Tabla 6. Modelo de cobro de CloudWatch.

Concepto	**Precio**	**Capa gratuita mensual**
Métricas que publican los servicios de AWS (`Errors`, `Invocations`, `Duration` de Lambda)	Sin costo	—
Métricas personalizadas	0.30 USD por métrica al mes	10 métricas
Alarma de resolución estándar (60 s)	0.10 USD por métrica de alarma al mes	10 alarmas
Alarma de alta resolución (10 s)	0.30 USD por métrica de alarma al mes	No aplica
Ingesta de logs	0.50 USD por GB (los de Lambda bajan por volumen hasta 0.05 USD por GB)	5 GB en total entre ingesta, almacenamiento y consultas
Almacenamiento de logs	0.03 USD por GB al mes	
Consultas con Logs Insights	0.005 USD por GB analizado	
Paneles (dashboards)	3.00 USD por panel al mes	3 paneles

Por eso tomamos tres decisiones: usar la métrica nativa `Errors` en lugar de una personalizada, crear una sola alarma estándar sin acciones de SNS y limitar la retención de logs a 3 días, para que el almacenamiento no crezca.

13.2. Cuánto nos costó

Tabla 7. Uso de CloudWatch del Equipo 3 durante la implementación y las pruebas (27 y 28 de octubre de 2026).

Concepto	Lo que usamos	Costo sin capa gratuita	Costo real
Métricas	Solo las nativas de Lambda	0.00 USD	0.00 USD
Alarmas	1 alarma estándar	0.10 USD al mes	0.00 USD
Logs	6.7 KB almacenados (figura 26), menos de 0.0001 GB ingeridos	menos de 0.01 USD	0.00 USD
Paneles y Logs Insights	No se usaron	0.00 USD	0.00 USD
Total de CloudWatch		**≈ 0.10 USD al mes**	**0.00 USD**

El gasto real fue de **0.00 USD**: todo quedó dentro de la capa gratuita, igual que Lambda, S3 y DynamoDB. La consola mostró 120.00 USD de créditos y 0.00 USD de gasto en el mes antes de empezar y al terminar (figuras 50 y 51). Aun sin capa gratuita, el costo sería de unos 10 centavos de dólar al mes, casi todo por la alarma. Como referencia, cada publicación genera alrededor de 1 KB de logs: una gaceta con 1 000 noticias al mes ingeriría cerca de 1 MB, unos 0.0005 USD.

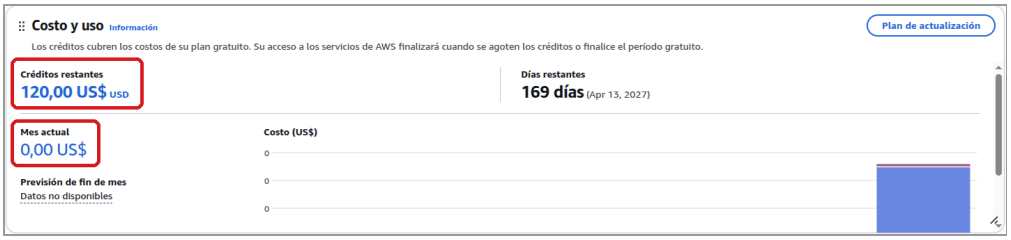


Figura 50. Al empezar (27 de octubre): 120.00 USD de créditos y 0.00 USD de gasto en el mes.

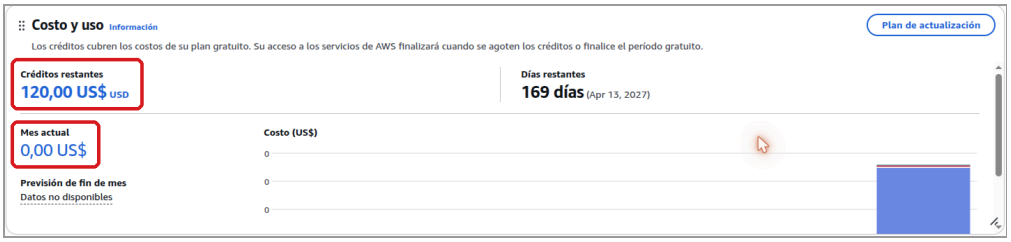


Figura 51. Al terminar las pruebas: los mismos 120.00 USD y 0.00 USD de gasto.

Una semana después seguía igual. En la página de facturación, los créditos restantes eran 120.00 USD y el importe a pagar 0.00 USD, es decir, octubre no consumió créditos. En **Cost Explorer**, agrupando por servicio y excluyendo créditos y reembolsos (lo que costaría sin créditos), CloudWatch marca 0.00 USD.

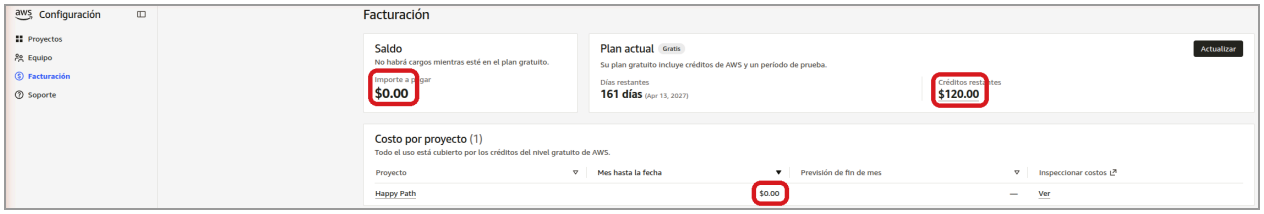


Figura 52. Configuración de AWS › Facturación (4 de noviembre): importe a pagar 0.00 USD, créditos restantes 120.00 USD y costo del mes 0.00 USD.

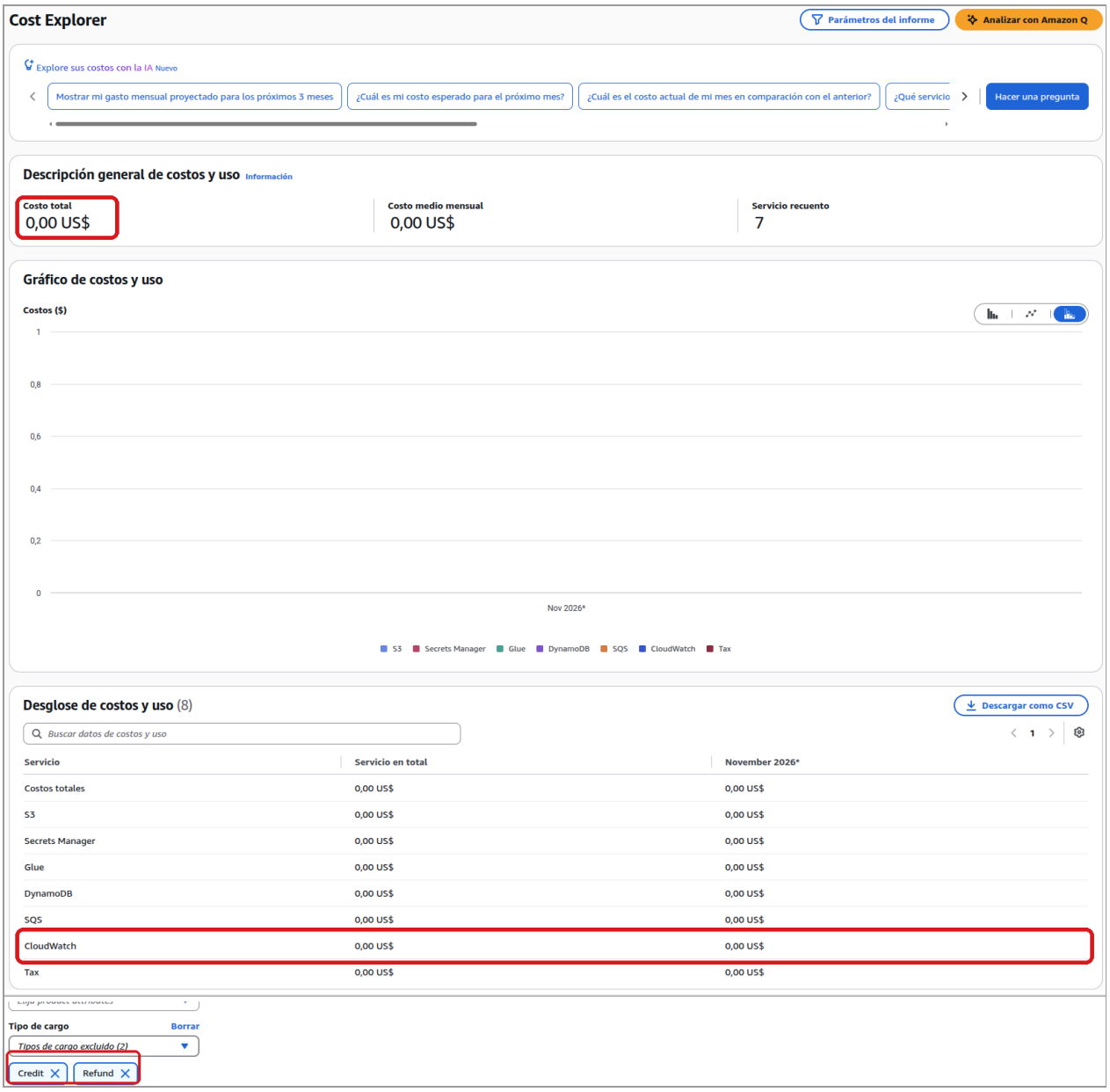


Figura 53. Administración de facturación y costos › Cost Explorer, agrupado por servicio, noviembre de 2026. Abajo, el filtro que excluye los tipos de cargo «Credit» y «Refund»: aun sin descontar créditos, CloudWatch cuesta 0.00 USD. El informe no mostró octubre.

14. Problemas frecuentes

Tabla 8. Síntomas y solución.

Síntoma	**Causa y solución**
El sitio sigue llamando a `agregarNoticia` al publicar con imagen	El navegador usa el `main.js` viejo de la caché. Cambia el `?v=` en `index.html`, súbelo y recarga con `Ctrl`+`F5`.
Error de CORS al enviar la noticia	Revisa la URL de función de prefirmadas: origen del sitio, encabezado `content-type` y método `POST`.
Falla la subida de la imagen (403 o CORS)	El CORS del bucket debe permitir `PUT`. La URL firmada vence a los 5 minutos.
«No puede haber prefijos superpuestos» al crear el disparador	Ya existe una regla sobre `imagenes/`. Revisa **S3 › Propiedades › Notificaciones de eventos**.
El log dice `AccessDenied` en lugar de `NoSuchKey`	Falta `s3:ListBucket` con la condición de prefijo en la política del rol.
El log dice `Bucket no esperado`	El evento de prueba o la variable `BUCKET` tienen otro nombre de bucket.
La noticia se queda «en procesamiento»	Busca `fallo` en los logs. Lambda reintenta dos veces durante una hora; al corregir la causa, el reintento la guarda.
No aparece el grupo de logs	Se crea con la primera ejecución: ejecuta una prueba en **Probar**.
La alarma se queda en «Datos insuficientes»	Configura «Tratar los datos que faltan como correctos».

15. Limpieza

Al terminar el curso, eliminaremos los recursos en este orden:

1. El desencadenador de S3 de la procesadora.
2. La función `procesar-imagen-noticia` y el rol `procesar-imagen-noticia-rol`.
3. La alarma `gaceta-procesadora-errores`.
4. El grupo de logs `/aws/lambda/procesar-imagen-noticia`.
5. Los objetos de `pendientes/`, las imágenes de prueba y las noticias de prueba de la tabla.

Si el sitio sigue en uso, antes hay que restaurar en `cors` el código original (`prefirmadas-original.mjs`) y el `main.js` anterior; si no, las noticias con imagen dejarían de guardarse.